Tunisian Republic

Ministry of Higher Education

and Scientific Research

End-of-Studies Project Report

Presented for obtaining the

National Engineering Degree

Specialty : Software Engineering

By

Amine Bébba

Ready to Deploy Enterprise-Specific LLM Development with Fine-Tuning and Retrieval-Augmented Generation

Professional Supervisor : **Mr. Yassine Delil**

Academic Supervisor : **Mrs. Aroua Hedhili**

Completed at Sopra HR Software

A.Y. : 2024-2025

I authorize the student to submit his internship report for defense.

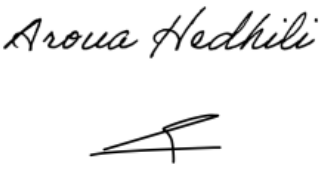
Professional Supervisor, **Mr. Yassine Delil**

Signature

I authorize the student to submit his internship report for defense.

Academic Supervisor, **Mrs. Aroua Hedhili**

Signature



Dedications

To all the people dear to my heart, To my dearest mother Ghayda Mandhour for all her unconditional love, infinite sacrifices, and dedication, and for being my guiding star through every challenge. Your unwavering faith in me has been my greatest strength, To my dearest sister Imen Bébba for her love, support, and precious presence, To my dear grandmother Najiba for her love and support, To my dear aunt Jafra for her love and generous support, To my dear uncles Wassim , Hama and Haithem for their support and being a resourceful reference to Technology , Electronics and Computer Science from an early age. In memory of my Father Mondher and Grandfather El-Hechmi who always believed in me to achieve this moment in life. To my lovely life companion, Oumaima, thank you for always supporting me, believing in me, and standing by my side in everything I do. To all my family and friends and all those who have never stopped believing in me, I dedicate this modest work,my efforts and many years of study and learning. May they find in it the crowning achievement of their assistance and the deep expression of my gratitude.

Acknowledgement

I would like to express my deepest gratitude to those who have supported and guided me throughout the completion of this project. Their contributions have been invaluable, and without their assistance, this work would not have been possible.

Primarily, I would like to thank Mrs. Aroua Hedhili, my academic supervisor, for his constant guidance, insightful feedback, and unwavering support. Their expertise and encouragement have been instrumental in the successful completion of this project.

I am immensely grateful to Mr. Yassine Delil, my professional supervisor, for his mentorship and for providing me with the opportunity to carry out this project. His practical insight and professional advice have enriched my understanding and execution of the project.

Thank you to all the individuals taking part of ProdOps R&D who have provided invaluable support, guidance, and encouragement throughout my academic journey and during the completion of this project.

Table of Contents

General Introduction **1**

1 Project Framework Overview **3**

1.1 Presentation of the Host Company 4

1.1.1 Presentation of Sopra HR Software 4

1.1.2 Team Description : Production Operations R&D – HR Access 5

1.1.3 Services 6

1.2 Project Contexte 6

1.3 Study of Existing Solutions 7

1.3.1 Analysis of the Current Solutions 7

1.3.2 Critique of the Existing Solutions 8

1.4 Problem Statement 8

1.5 Proposed Solution 9

1.6 Methodological choice 10

1.7 Conclusion 12

2 Business Understanding **14**

2.1 Business Background 15

2.1.1 Jira Support Ticket Management 15

2.1.2 Linux Server Administration 15

2.1.3 Environment Maintenance and Duplication 16

2.1.4 Technical Migrations 17

2.1.5 HR Access Release Upgrades and Tomcat Server Updates 17

2.1.6 Troubleshooting and Diagnostics 17

2.1.7 Performance Monitoring 18

2.1.8 Operational Continuity and Innovation 18

2.2 Business Objectives 18

2.3 Project Goals 18

2.4 Success Criteria (Business KPIs) 19

2.5 Conclusion 20

3 Data Understanding and Preparation **21**

3.1 Data Understanding 22

3.1.1 Data Inventory 22

3.1.2 Data Characteristics 23

3.1.3 Initial Analysis and Exploration 24

3.2 Data Preparation 24

3.2.1 Data Collection 25

3.2.2 Data Cleaning 26

3.2.3 Data Transformation 27

3.3 Conclusion 28

4 Modeling **30**

4.1 What is an LLM and Why Pretrained Models ? 31

4.2 Model Choice 31

4.2.1 Why Is Model Selection Important ? 31

4.2.2 Key Steps in Model Selection 32

4.2.3 Requirements and Constraints 33

4.2.4 Candidate Models 33

4.2.5 Final Choice and Justification 34

4.3 Model Fine-Tuning 35

4.3.1 Prompt Engineering for Supervised Training 35

4.3.2 Efficient Fine-Tuning with Unsloth and LoRA 36

4.3.3 Supervised Fine-Tuning with `SFTTrainer` 37

4.3.4 Experiment Tracking with Weights & Biases 38

4.3.5 Optimisation, Quantisation, and Practical Impact 39

4.3.6 LLM Fine-Tuning architecture overview 40

4.4 RAG : Retrieval Augmented Generation 40

4.4.1 Importance of RAG for Enterprise LLMs 41

4.4.2 RAG Components and Pipeline 41

4.4.3 RAG Architecture Overview 43

4.5 RAFT : Retrieval Augmented Fine Tuning RAFT = RAG + FT 44

4.5.1 RAFT : State of the Art 44

4.5.2 RAFT Architecture Overview 45

4.6 Conclusion 46

5 Evaluation and Deployement **47**

5.1 Evaluation Objectives and Methodology 48

5.2 Test Environment 49

5.2.1 Overview of the Kaggle Environment 49

5.2.2 Software Configuration 50

5.2.3 Hardware Configuration 50

5.2.4 Reproducibility and Runtime Stability 51

5.3 Pre-Trained LLM Evaluation for Model Selection 51

5.3.1 Manual Inference and Qualitative Comparison 51

5.4 Fine-Tuned LLM Evaluation 52

5.4.1 Weights and Biases Metrics 52

5.4.2 General Technical Knowledge Inference 54

5.4.3 Specific Enterprise Knowledge Inference 54

5.5 RAG : Context Retrieval Evaluation 55

5.5.1 Semantic Retrieval Evaluation 55

5.5.2 Lexical Retrieval (BM25) 55

5.5.3 Hybrid Retrieval and Fusion 56

5.6 RAFT : Full Project Outcome Evaluation 56

5.6.1 Performance and Impact Metrics 56

5.6.2 Deployment and Go/No-Go Recommendations 56

5.7 Ready To Deploy Model 57

Conclusion et Perspectives **59**

Bibliographic References **61**

List of Figures

1.1 CRISP-DM Diagram 12

3.1 Jira Scraper output files 25

3.2 Jira error–solution ticket extraction pipeline 26

3.3 Pipeline for preprocessing `jira_tickets.txt` into structured error–solution pairs . . . 27

3.4 Pipeline for extracting error–solution pairs and formatting them as an LLM training dataset 28

4.1 LLama 3 8B - Mistral 0.3 7B Benchmark 34

4.2 Training Prompt Design 35

4.3 LoRA Algorithm and Used Parametres with Respective Description 37

4.4 SFTTrainer Algorithm and Used Parametres with Respective Description 38

4.5 W&B Graphs 39

4.6 Pipeline for Model Fine Tuning 40

4.7 Pipeline for Hybrid Retrieval 43

4.8 Pipeline for RAG and LLM inference 43

4.9 Global Architecture 46

5.1 Kaggle Environment Interface 51

5.2 Inference Test Results 52

5.3 Weights and Biases Metrics Monitoring 53

5.4 Hugging Face Model Packaged 57

5.5 Model Import Snippet 58

List of Tables

1.1 Comparison of development methodologies for AI projects 11

Liste des abréviations

[AI] **A**rtificial **I**ntelligence [LLM] **L**arge **L**anguage **M**odel [RAG] **R**etrieval **A**ugmented **G**eneration [HRIS] **H**uman **R**esources **I**nformation **S**ystem [GPU] **G**raphics **P**rocessing **U**nit [VRAM] **V**ideo **R**andom **A**ccess **M**emory [LoRA] **L**ow **R**ank **A**daptation [PEFT] **P**arameter **E**fficient **F**ine **T**uning [SFT] **S**upervised **F**ine **T**uning [RLHF] **R**einforcement **L**earning from **H**uman **F**eedback [BM25] **B**est **M**atch **25** (lexical ranking function) [TF-IDF] **T**erm **F**requency–**I**nverse **D**ocument **F**requency [API] **A**pplication **P**rogramming **I**nterface [JSON] **J**ava**S**cript **O**bject **N**otation [PDF] **P**ortable **D**ocument **F**ormat [SQL] **S**tructured **Q**uery **L**anguage [CI/CD] **C**ontinuous **I**ntegration / **C**ontinuous **D**eployment [MLOps] **M**achine **L**earning **Op**eration**s** [LLMOps] **L**arge **L**anguage **M**odel **Op**eration**s** [Ops] **Op**eration**s** [SRE] **S**ite **R**eliability **E**ngineering [HPC] **H**igh **P**erformance **C**omputing [VRAM] **V**ideo **R**andom **A**ccess **M**emory [NLP] **N**atural **L**anguage **P**rocessing [QPS] **Q**ueries **P**er **S**econd [TTFT] **T**ime **T**o **F**irst **T**oken [DPS] **D**ocuments **P**er **S**econd [KPI] **K**ey **P**erformance **I**ndicator [ROI] **R**eturn **O**n **I**nvestment [W&B] **W**eights & **B**iases [HF] **H**ugging **F**ace [T4] **NVIDIA T**esla **T**4 **GPU** [H100] **NVIDIA H**100 **GPU**

General Introduction

Artificial Intelligence (AI) is rapidly transforming the way the world operates, influencing virtually every sector from healthcare and finance to manufacturing, education, and scientific research. In recent years, breakthroughs in AI technologies particularly large language models (LLMs) have unlocked powerful capabilities in natural language understanding, reasoning, and problem-solving. With the constant stream of cutting-edge research papers, new architectures, and innovations such as Retrieval-Augmented Generation (RAG), multimodal learning, and agent-based systems, AI is evolving at an unprecedented pace.

This evolution is not merely academic ; it is driving tangible benefits across industries. Enterprises are increasingly leveraging AI to enhance productivity, improve decision-making, reduce operational costs, and automate repetitive tasks. LLMs, when fine-tuned and customized for specific enterprise contexts, become not only general-purpose tools but also domain-aware digital assistants that understand internal workflows, technical terminology, and organizational goals. These models can draft technical documentation, extract insights from vast datasets, support RD with real-time knowledge synthesis, and assist experts in highly specialized tasks—from problem-solving to regulatory compliance. By integrating LLMs into daily operations, companies are enabling smarter collaboration, faster innovation, and more agile responses to market and technological changes.

In a context marked by accelerated digital transformation, all organizations must reinvent their processes to gain agility, efficiency, and responsiveness. This transformation affects all domains, including those traditionally perceived as strictly administrative or functional. Digital tools are no longer limited to conventional business applications but now extend to more intelligent solutions.

It is within this framework that Sopra HR Software, an internationally recognized HR solutions provider, initiated an internal project within its Production Operations Research and Development (RD) team. This project involves developing a domain-specific Large Language Model (LLM) combining fine-tuning and Retrieval-Augmented Generation (RAG) techniques, trained on proprietary HR Access software data including error logs, documented solutions, troubleshooting procedures, and advanced operational workflows.

The objective is to create an internal AI-powered expert system that significantly improves consultants' productivity by providing precise, contextually relevant solutions to complex technical queries. Through parameter-efficient fine-tuning methods like LoRA, the LLM learns HR Access-specific terminology and error patterns, while the RAG component retrieves up-to-date documentation and historical solutions in real-time. This hybrid approach enables the system to handle both learned

domain expertise and dynamic knowledge retrieval, reducing consultants' dependence on manual searches and senior escalations.

A critical advantage is the system's continuous evolution capability. The infrastructure supports ongoing retraining with new error cases and documentation updates, while the RAG component seamlessly integrates newly added knowledge without requiring full model retraining. This ensures the LLM remains current with evolving software versions and organizational practices, effectively capturing and preserving institutional knowledge.

To structure this work, the **CRISP-DM**[1] was selected, particularly suited for projects combining textual data and artificial intelligence. It provides a clear approach and ensures no critical steps are skipped.

The report is organized as follows. The first chapter presents the business context and project needs. The data collection, analysis, and cleaning phase follows, ensuring quality training inputs. Next, the pretrained model selection and fine-tuning process are explained, including parameter-efficient methods. The subsequent chapter covers the integration of an advanced RAG system with a hybrid search algorithm for optimal retrieval. The evaluation phase tests system performance through accuracy metrics and real-world scenarios. Finally, model packaging and preparation for deployment-ready release on Hugging Face Hub is addressed, including model serialization, documentation, and inference configuration, followed by conclusions and future recommendations.

The objective of this report is therefore to demonstrate how, from a concrete team need, a functional solution was built using both academic knowledge and hands-on learning acquired during the internship. As AI capabilities continue to expand, its role in shaping the future of work, research, and innovation becomes more central—offering enterprises a unique opportunity to harness this momentum for competitive advantage and transformative growth

Chapter 1

Project Framework Overview

Plan

1 Presentation of the Host Company 4

2 Project Contexte 6

3 Study of Existing Solutions 7

4 Problem Statement 8

5 Proposed Solution 9

6 Methodological choice 10

7 Conclusion 12

Introduction

This section introduces the host organization, Sopra HR, and defines the project framework, context, and objectives. It also aims to position the work within its professional and organizational environment. Understanding how the company operates and the reasons behind the project is essential for interpreting the technical and functional choices presented in the following chapters..

1.1 Presentation of the Host Company

Sopra HR Software, a subsidiary of the Sopra Steria Group, is a leading provider of human resources and payroll management solutions, serving large organizations across both public and private sectors. With over five decades of experience and a presence in more than 50 countries, the company supports millions of employees worldwide through its flexible, scalable HR platforms. Sopra HR offers a comprehensive suite that includes payroll, core HR management, talent development, time tracking, and employee self-service delivered through cloud, on-premises, or outsourced service models. The company is committed to helping organizations modernize their HR processes while improving employee experience and operational efficiency. Backed by continuous investment in innovation, Sopra HR integrates emerging technologies such as AI, RPA, and advanced analytics into its offerings, positioning itself as a strategic partner for digital HR transformation.[2]

History

Sopra HR Software has its roots in the broader legacy of the Sopra Steria Group, a European leader in digital transformation. With over 50 years of expertise in information systems and enterprise solutions, Sopra Steria created Sopra HR as a dedicated subsidiary to specialize in human resources and payroll technologies. Over the decades, Sopra HR has evolved from providing traditional payroll systems into a major player in comprehensive HR management solutions. The company has grown through continuous innovation, international expansion, and strategic partnerships, enabling it to support complex HR environments for large organizations across Europe, Africa, and beyond. Its strong foundation in both IT services and HR domain knowledge positions it as a trusted partner for public institutions and global enterprises alike. [2]

1.1.1 Presentation of Sopra HR Software

Sopra HR Software is a global provider of human resources solutions and a subsidiary of the Sopra Steria Group. Specializing in HR and payroll systems, Sopra HR supports public and private organizations in their digital transformation of the HR function. Through its software and services, the

company helps HR departments manage payroll, personnel administration, time and activity tracking, talent management, and HR analytics, while enhancing the employee experience. As part of the Sopra Steria Group, Sopra HR benefits from strong technological expertise and an international presence, enabling it to deliver scalable, innovative solutions tailored to complex and large-scale environments.

1.1.2 Team Description : Production Operations R&D – HR Access

Overview The Production Operations R&D team occupies a pivotal technical role throughout the lifecycle of the HR Access product within Sopra HR Software. This group is charged with the design, management, and maintenance of the entire ecosystem of R&D environments, which are deployed on both on-premise Linux servers and Amazon Web Services (AWS) cloud infrastructure. Their efforts are foundational to the continuity, security, and optimization of all development, testing, and production activities.

Key Responsibilities

- **Environment Management :** Administration and performance optimization for all R&D environments supporting HR Access, encompassing both traditional Linux server deployments and scalable AWS platforms.
- **Server Configuration :** Specification, configuration, and deployment of server resources, including selection of appropriate hardware and virtualized/cloud instances, network setup, storage allocation, and OS-level tuning. Consistent application of security hardening and compliance standards.
- **Database Management :** Operation, tuning, and maintenance of Oracle SQL databases for high availability, robust performance, backup and disaster recovery, and data integrity.
- **System Monitoring & Reliability :** Continuous, proactive surveillance using monitoring solutions to detect performance bottlenecks, preempt failures, and rapidly diagnose incidents across the environments.
- **Upgrades & Patching :** Planning and execution of product upgrades, patch application, and regular updates to maintain security, ensure compatibility, and support new features.
- **Duplication & Deployment :** Execution of environment duplication, data refresh, configuration replication, and streamlined deployment workflows underpinning both R&D and production lifecycles.
- **Problem Investigation :** Systematic investigation of software, infrastructure, and integration issues using advanced diagnostic tools, log analysis, and tracing techniques.

— **Solution Implementation :** Design and deployment of permanent fixes, preventive measures, and process improvements based on root cause analyses, ensuring continuous improvement in operation stability.

— **Technical Operations & Support :** Comprehensive management of all intermediate and advanced technical operations necessary to keep HR Access performant, up-to-date, and fully aligned with industry standards and security requirements.

Impact and Contribution Through their deep and continuous technical involvement, the Production Operations R&D team serves as a backbone for HR Access platform stability and innovation. The team's actions directly enhance reliability, support product evolution, and underpin efficient R&D and delivery processes, thereby enabling Sopra HR to offer high-quality services to its enterprise clients.

1.1.3 Services

Sopra HR Software delivers a broad range of human capital management services designed to meet the diverse needs of large and mid-sized organizations. Its solutions cover the entire HR value chain, including core HR administration, payroll processing, time and attendance tracking, talent management, and workforce planning. Through its modular and scalable platforms available in SaaS, cloud, or on-premises models Sopra HR helps companies streamline operations, ensure compliance, and improve employee engagement. Additionally, the company offers Business Process Outsourcing (BPO) services, taking full responsibility for managing payroll and HR processes on behalf of clients.

1.2 Project Contexte

This end-of-studies project was carried out within the Production Operations RD team at Sopra HR Software, with the aim of introducing AI into the team's daily workflow. The work focuses on building a domain-specific Large Language Model trained on internal HR Access data, particularly error logs, documented fixes, and detailed procedure guides.

The goal is to provide the team with a fast and reliable way to get answers to their questions, especially those related to complex errors and advanced operational steps. Thanks to model optimization and careful design, the LLM is intended to act as an internal expert : quickly suggesting relevant solutions, guiding consultants through intricate procedures, and helping them save time while improving the precision and consistency of their interventions. Over time, the solution is designed to evolve as new data and documentation are added, strengthening the team's collective expertise.

1.3 Study of Existing Solutions

It is essential to analyze the current situation, both in terms of practices and the limitations of the tools currently in use. This approach makes it possible to better identify the real needs in intensive shrimp farming and the challenges involved in implementation. In light of the results of this analysis, it then becomes relevant to propose a solution that addresses the identified issues and the operational constraints encountered in the field.

1.3.1 Analysis of the Current Solutions

Manual log debugging Consultants frequently rely on manual log debugging, searching for error messages directly in server logs and cross-checking them with personal notes or team knowledge. This process is highly manual and depends heavily on individual experience and memory.

Manual solutions lookup from Jira When an error occurs, the team often searches old Jira tickets in the hope of finding a similar issue and its corresponding resolution. This requires formulating multiple search queries, scanning through long ticket threads, and inferring whether a past solution is applicable to the current context.

Manual documentation search Another common practice is to consult internal documentation (wikis, knowledge bases, functional or technical guides) to retrieve information that might help with troubleshooting. This search is usually keyword-based and can span several documents before finding actionable content.

Internet search Some errors appear to be generic or product-agnostic, which leads consultants to search on the internet (forums, blogs, vendor documentation) for potential explanations or fixes. While this can occasionally help, the results are not tailored to Sopra HR's internal configurations or client-specific contexts.

Use of general-purpose LLMs (e.g., ChatGPT) General-purpose LLMs are sometimes used to clarify concepts, interpret error messages, or outline possible troubleshooting paths. However, since they have no direct access to proprietary HR Access data, their answers remain generic and are often insufficient for enterprise-specific issues.

1.3.2 Critique of the Existing Solutions

The existing troubleshooting methods share several limitations. First, they are time-consuming : each approach (log inspection, Jira searches, documentation review, or internet lookup) requires multiple iterations before converging on a relevant solution. Second, they are inconsistent and highly dependent on the individual doing the search ; two consultants facing the same problem may follow different paths and obtain different outcomes.

Moreover, these methods often fail to produce results when the issue is specific to Sopra HR's internal environments, custom configurations, or newly released versions. External resources and generic LLMs cannot reliably handle enterprise-specific edge cases, and existing internal documentation or tickets may be incomplete, hard to locate, or not indexed in a way that supports efficient retrieval. As a result, knowledge remains fragmented across tools and people, and recurring problems must sometimes be rediscovered and re-solved from scratch, which slows down workflows and reduces overall productivity.

1.4 Problem Statement

Within enterprises around the world, and specifically within the Sopra HR Software Production Operations R&D team, we are at the forefront of testing and interacting with newly released versions of the product, multiple environments, and a wide array of enterprise modules. As early adopters and first-line users, we consistently encounter issues that have yet to be documented, resolved, or even identified elsewhere. These problems span a wide spectrum, ranging from recurring environment-specific issues and version-related bugs to configuration errors and low-level technical incompatibilities such as discrepancies in Java or Tomcat versions.

The nature of this work places the team in a highly dynamic and technically complex landscape where new problems emerge frequently and require timely, context-aware solutions. Many of these issues are not one-off occurrences ; they reappear across environments or evolve with new deployments, amplifying both the volume and variety of technical challenges faced. This results in a rapidly growing pool of knowledge that must be continuously created, refined, and referenced.

Currently, the troubleshooting process is heavily reliant on manual information retrieval : digging through fragmented internal documentation, scouring online resources, or relying on personal notes and past experience. This not only makes problem-solving time-consuming, but also inefficient and inconsistent, especially when team members change or when issues recur in slightly different contexts. The lack of a centralized, intelligent system for capturing, retrieving, and suggesting solutions exacerbates these challenges, ultimately slowing down workflows and reducing overall productivity.

As the technical complexity and scale of the environment grow, so does the need for smarter, context-aware support systems capable of helping the team diagnose, understand, and resolve issues faster and more reliably.

1.5 Proposed Solution

At its core, the product LLM will be referred to as **Astrox** is designed to address three fundamental needs :

- **Expert assistance :** Act as a highly knowledgeable assistant for R&D and production operations tasks, with interactions that feel close to consulting a trusted, senior colleague.
- **Availability :** Provide fast, reliable access to expertise with minimal latency and no dependency on specific individuals' availability.
- **Expandability :** Continuously evolve by incorporating new data, documentation, and resolved cases, ensuring the model remains aligned with the latest product changes and practices.

Key Features

1. Domain-specific fine-tuning Astrox is fine-tuned on proprietary enterprise data, such as technical documentation, configuration guides, tickets, logs, and incident reports. This allows the model to understand internal terminology, product architecture, and workflows, and to adapt to the tone and structure of internal communication.

2. Retrieval-Augmented Generation (RAG) Astrox integrates an advanced RAG layer that dynamically retrieves relevant documents, notes, or code snippets from internal knowledge bases at query time. This grounding in real, up-to-date enterprise knowledge increases the reliability of answers and reduces hallucinations. It also allows the system to handle fast-changing environments, such as frequent version upgrades or configuration changes, more effectively than static models.

3. Context-aware problem solving The system supports multi-step reasoning, debugging assistance, and root-cause analysis by combining current and historical data. It can take into account environment details, version information, and past incidents to propose more accurate and actionable suggestions.

4. Integration with internal tools Astrox is designed to integrate naturally with internal systems such as Jira, wikis, or internal portals. This enables it to reference and enrich existing workflows instead of duplicating them.

5. Multi-lingual and multi-modal support The model can process and generate content across different natural and programming languages. It is capable of analyzing logs, stack traces, error messages, and potentially structured artifacts such as configuration files or architecture diagrams.

6. Security and access control The solution is aligned with enterprise-grade security practices, including data isolation, access restrictions, and role-based retrieval. Sensitive information is only accessible to authorized users through fine-grained permission mechanisms.

7. Scalability and performance Astrox is optimized to handle large volumes of queries across extensive document collections. It is designed to be deployable in a private cloud or on-premise environment to meet compliance, security, and performance requirements, while remaining ready to be published as a deployment-ready model on platforms such as Hugging Face Hub.

1.6 Methodological choice

In this project, it is essential to define a rigorous methodology suited to the goals of data analysis and value creation. Among structured approaches developed to guide data science projects, this work adopts CRISP-DM (Cross-Industry Standard Process for Data Mining). This methodology provides a clear, stage-based process aligned with the project's nature, available tools, and business constraints, and the following presents an overview tailored to our context using CRISP-DM

Criterion / Methodology	**V-Model cycle_v**	**Agile scrum_agile**	**KDD kdd_process**	**CRISP-DM crispdm**
Orientation	Traditional software	Software / Product	Data Mining	Data Mining / AI
Structure	Sequential	Iterative, incremental	Sequential with loops	Iterative, flexible
Data handling	Poorly suited	Poorly suited	Central	Central
Adaptation to change	Low	Excellent	Medium	Excellent
Key phases	Specification, dev, test	Backlog, sprints, demo	Selection, preprocessing, mining	Business understanding, data understanding, preparation, modeling, evaluation, deployment
Traceability	High	Medium	Medium	High
Business involvement	Low	High	Medium	High
AI/Data specificity	No	No	Yes	Yes
Popularity in AI	Low	Medium	Medium	Very high

Tableau 1.1 : Comparison of development methodologies for AI projects

1.6.0.1 Justification of CRISP-DM

This project is built around the exploitation and structuring of complex data to power an intelligent, AI-based assistant. Among the different methodologies considered, **CRISP-DM** (Cross-Industry Standard Process for Data Mining) stands out as the most suitable for this type of work, as it was specifically designed for projects where data is central, unlike more traditional software-oriented approaches such as the V-Model or generic agile methods.

CRISP-DM structures a data project into six main phases :

— **Business understanding** : clarify the business context, objectives, constraints, and success criteria of the project.

— **Data understanding** : collect the initial data, explore its structure and quality, and identify patterns or anomalies.

— **Data preparation** : clean, transform, and select the relevant data needed to build reliable models.

— **Modeling** : choose appropriate modeling techniques, train the models, and tune their parameters.

— **Evaluation** : assess the models against technical metrics and business goals to verify that they truly address the problem.

— **Deployment** : integrate the selected model into a usable solution (or deployment-ready artifact), and plan how it will be maintained and monitored over time.

Beyond this clear structure, CRISP-DM encourages iteration and continuous improvement, which is crucial in a context where both data and business needs evolve over time. It also reinforces the traceability of technical choices and requires each phase to be validated before moving on to the next.

In this project, adopting CRISP-DM has helped secure the overall development process, organize data preparation in a systematic way, frame the LLM modeling work, and ensure rigorous evaluation of the results. Its wide adoption in the data science industry and its flexibility in real-world environments make it a particularly well-aligned methodology for this project.

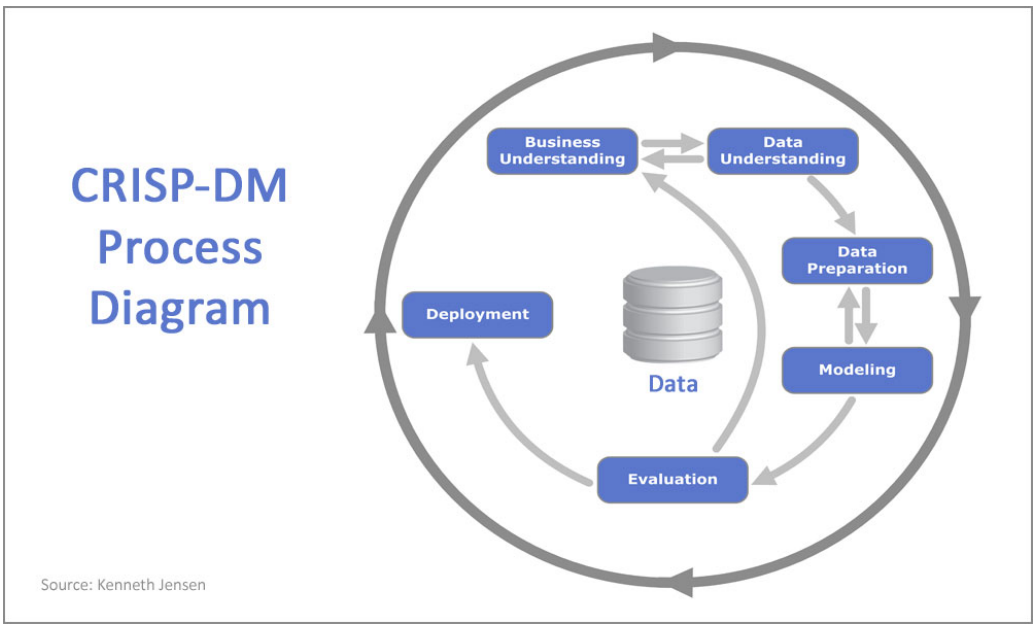


Figure 1.1 : CRISP-DM Diagram

1.7 Conclusion

In ProdOps RD, the growing complexity and variability of technical issues make manual knowledge search inefficient and fragmented. An enterprise-specific, fine-tuned LLM combined with RAG can serve

as an intelligent copilot that understands the domain, retrieves the right context, and supports complex technical reasoning. This not only speeds up troubleshooting but also boosts team productivity, preserves and enriches organizational knowledge, and adds a strategic, AI-driven layer of operational intelligence that improves continuously.

Chapter 2

Business Understanding

Plan

1 Business Background 15

2 Business Objectives 18

3 Project Goals 18

4 Success Criteria (Business KPIs) 19

5 Conclusion 20

Introduction

Understanding the business context is the essential first step in any data driven project. In the CRISP-DM methodology, the Business Understanding phase is designed to clarify the real world motivations, challenges, and goals that shape all subsequent project work. For this project, gaining deep insight into the structure, needs, and pain points of the Production Operations R&D team at Sopra HR Software is vital ensuring that the proposed AI solution directly addresses critical operational objectives and brings measurable value to the organization.

This chapter summarizes the business background, defines the key challenges faced by the team, sets out the intended goals of the project from a user and company perspective, and outlines the key metrics by which success will be measured. Framing the problem in this way lays a strong foundation for the technical work that follows, and ensures that each step remains aligned with Sopra HR Software's broader strategic priorities.

2.1 Business Background

Sopra HR Software, a subsidiary of the Sopra Steria Group, provides advanced Human Resource solutions for large organizations and public sector entities. Within Sopra HR, the Production Operations R&D team plays a critical role in maintaining, optimizing, and evolving HR Access environments to guarantee system stability, performance, and innovation.

2.1.1 Jira Support Ticket Management

A central aspect of the team's daily routine involves managing support tickets through Jira. These tickets may include environment requests, such as provisioning new test systems or duplicating existing ones, as well as incident tickets relating to service disruptions, bug reports, or urgent troubleshooting needs. The team works through a steady queue of tickets each day, prioritizing tasks, communicating with stakeholders, and ensuring timely resolution. This process is a critical part of coordinating operational activities, maintaining transparency, and systematically addressing both immediate incidents and strategic requests.

2.1.2 Linux Server Administration

The team is responsible for end-to-end management of all Linux servers that underpin application and database environments crucial to HR Access operations. This involves configuring optimal system parameters and implementing robust security standards to ensure system integrity.

Key technical responsibilities include :

— **Storage Management :** Monitoring disk usage and partitioning, managing quotas to prevent resource exhaustion, configuring and mounting file systems, and regularly cleaning up and archiving old or unnecessary files to optimize performance and maintain sufficient free space.

— **Swap and Memory Optimization :** Setting appropriate swap sizes based on workload demands, and monitoring memory utilization to ensure smooth operation under variable loads.

— **CPU and Performance Tuning :** Tracking CPU usage, identifying high-load processes, and tuning kernel parameters for optimal server responsiveness and throughput.

— **Resource Monitoring :** Using tools and scripts to track real-time and historical trends in server health covering disk I/O, CPU, memory, and network usage so that potential bottlenecks or failures can be detected and addressed proactively.

— **Backup and Disaster Recovery :** Implementing backup strategies, scheduling regular data snapshots, and periodically testing restore processes to safeguard business continuity.

By combining these practices, the team maintains high availability, performance, and security for all HR Access environments, supporting both day-to-day reliability and long-term operational goals.

2.1.3 Environment Maintenance and Duplication

Production Operations R&D oversees a variety of environment maintenance activities to ensure the stability and availability of HR Access systems. Key technical responsibilities include :

— **Backup and Archiving :** Prior to any duplication or major change, the team creates full backups of the existing environment, encompassing both the application files and the Oracle database, to safeguard against data loss and enable reliable rollback.

— **Environment Duplication (Application and Database) :** The duplication process begins with the export of the application instance and a dump of the Oracle database from the source environment. The exported application files and database backup are then transferred to the target environment or server.

— **Restoration and Reconfiguration :** In the target environment, the Oracle database is imported and the application is deployed. Configuration files such as those governing port numbers, service addresses, and Tomcat (Catalina) settings are carefully adapted to match the new environment's specifics.

— **Service Validation and Testing :** After imports and configuration changes, the team conducts a thorough test of critical services and endpoints. Verification commonly covers the proper booting of core components like the ADM server, the OpenHR web portal, and services delivered

through `tomweb` and `tombatch`. This ensures all services are accessible and fully functional in the duplicated environment.

— **Troubleshooting and Finalization :** Any startup or runtime issues are diagnosed and resolved, logs are reviewed, and the environment is validated for handoff to users or for subsequent development and testing activities.

Through these technical procedures, the team provides reliable test, training, and backup systems, supporting continuous development and minimizing operational risks for the organization.

2.1.4 Technical Migrations

The team manages complex technical migrations which often involve duplicating an existing HR Access environment to a new server running a different operating system. This process includes backing up the original environment's application files and Oracle database, exporting them, and importing into the new environment.

Since the new server OS differs, specialized generator programs need to be compiled to ensure the HR Access product runs correctly in the new environment. Configuration files, including those for Catalina services, require updates to reflect new ports, addresses, and service endpoints.

After migration, rigorous testing is conducted on critical components such as the ADM server, OpenHR, tomweb, and tombatch services to verify stability and performance before the environment is certified for use.

2.1.5 HR Access Release Upgrades and Tomcat Server Updates

Separately from migrations, the team is responsible for applying HR Access release upgrades. This involves planning the upgrade steps, validating new features and compatibility, and executing database and application updates with minimal disruption.

Tomcat server updates are also managed carefully to maintain web server performance and security. Service restarts and configurations are tested to ensure smooth integration with the upgraded software stack.

2.1.6 Troubleshooting and Diagnostics

The group actively monitors system health, analyzes error logs, and troubleshoots technical problems as they arise. They implement solutions for software and infrastructure issues, document resolutions, and share best practices across the organization.

2.1.7 Performance Monitoring

Production Operations R&D systematically tracks the performance and usage metrics of all managed environments, enabling proactive identification of bottlenecks and optimization opportunities. They use monitoring tools to analyze server statistics and ensure service level objectives are consistently achieved.

2.1.8 Operational Continuity and Innovation

Beyond day-to-day management, the team ensures operational continuity by quickly responding to technical and organizational challenges. Their contribution enables Sopra HR Software to adopt new technologies, support evolving customer requirements, and maintain stability across diverse HR Access deployments.

2.2 Business Objectives

Despite the team's deep expertise and dedication, they are repeatedly challenged by an unending cycle of complex technical errors, slow and often exhausting troubleshooting routines, and documentation that is fragmented, frequently outdated, and difficult to search. The emergence of new issues often buried in cryptic logs can transform even routine support into a painstaking scavenger hunt, draining valuable time and focus.

This constant struggle to isolate root causes and find the right fixes not only hampers productivity but is also mentally taxing, leading to frustration across the team. The lack of a clear, efficient pathway to the perfect solution for each incident means hours are spent piecing together scattered information or retracing steps already taken by others.

In such a demanding environment, the search for a more intelligent, streamlined approach is urgent. A truly effective problem-solving solution would bring clarity to chaos : enabling engineers and administrators to swiftly diagnose, resolve, and learn from even the most elusive and enterprise-specific errors without being held hostage by lengthy manual research or incomplete documentation. For Sopra HR Software, achieving this next level of support is essential not just for reducing downtime and increasing efficiency, but also for restoring calm and confidence to every troubleshooting mission.

2.3 Project Goals

The primary aim of this project is to deliver an LLM-powered assistant that is a true force multiplier for the team. Designed with both innovation and long-term value in mind, this solution transforms the way technical problems are addressed offering rapid, precise, and expert-level support

that feels almost human.

Unlike generic or resource-hungry AI, this system has been rigorously optimized for both hardware and software efficiency. By targeting reliable performance on cost-effective, low-end GPUs, the solution remains financially sustainable and maintainable as needs scale. On the software side, every aspect is engineered for speed and responsiveness : advanced retrieval and reasoning workflows provide targeted, actionable answers to complex errors in seconds.

This approach empowers staff with instant expertise that continually updates with new knowledge, dramatically reducing time lost to manual research or repetitive documentation reviews. Troubleshooting paths for even the most intricate and enterprise-specific issues are made transparent, actionable, and easy to follow minimizing stress and downtime across the team.

Ultimately, this assistant is more than a tool ; it's a strategic asset that embodies the spirit of AI-driven transformation at Sopra HR Software : fast, affordable, accurate, and always ready to support tomorrow's challenges.

2.4 Success Criteria (Business KPIs)

Project success for this initiative is defined by much more than workflow efficiency it is measured by the sustained technical excellence and user impact delivered by the RAG-powered assistant. The ideal outcome hinges on several core factors :

— **Performance and Accuracy :** The system should achieve high marks on critical retrieval and generation metrics, consistently delivering precise, relevant answers to a broad range of technical queries. Benchmarks such as accuracy, context adherence, and response latency will be closely monitored to ensure best-in-class results.

— **Resource Optimization :** Thanks to its lean architecture, the assistant is optimized for low compute resource consumption even running smoothly on cost-effective, low-end GPUs. This strongly supports long-term maintainability and makes it highly cost-efficient as the knowledge base and user base grow.

— **Adaptability and Ease of Maintenance :** A standout success metric is the assistant's ability to quickly digest updated documentation. The retraining pipeline is designed for simplicity and speed, so the model can be updated regularly with minimal manual effort, keeping its guidance current and trustworthy.

— **User Feedback :** Real-world value will be validated through positive feedback from daily users, including field engineers and administrators. Their experiences, collected through surveys or direct testimonials, will indicate the assistant's helpfulness in resolving issues and supporting

operational needs.

2.5 Conclusion

In summary, this chapter has revealed the complexities and critical requirements facing the Production Operations R&D team at Sopra HR Software. Their mission maintaining seamless, secure, and high-performing HR Access environments demands a support solution capable of meeting both technical and human challenges head-on.

The proposed LLM-powered assistant, optimized for cost-efficiency and technical excellence, sets a new benchmark in enterprise problem-solving. By leveraging a performant Retrieval-Augmented Generation (RAG) architecture, the system can not only digest and dynamically update its knowledge from internal documentation, but also deliver fast, accurate, and context-aware support, even on resource-constrained hardware. This positions the assistant as both sustainable and future-ready.

True project success will be measured through high-performance retrieval metrics, consistently positive user feedback, and ease of ongoing updates ensuring that every interaction is cost-effective, relevant, and impactful. Ultimately, the assistant's real value lies in its capacity to streamline knowledge flow, minimize downtime, empower users, and continuously adapt to the evolving landscape of Sopra HR's enterprise operations.

Chapter 3

Data Understanding and Preparation

Plan

1 **Data Understanding** 22

2 **Data Preparation** 24

3 **Conclusion** 28

Introduction

The Data Understanding and Preparation phases form a critical foundation of the CRISP-DM methodology, shaping all subsequent stages of the project. This chapter systematically inventories, analyzes, and refines the diverse data sources that will inform and train the LLM-powered assistant. By examining the variety, volume, and quality of operational logs, incident tickets, and technical documentation, it identifies both opportunities and challenges within the organization's data landscape. Building upon these insights, the preparation process focuses on cleaning, consolidating, and transforming raw information into structured, high-quality datasets—ensuring that the model is grounded in reliable, representative, and business-relevant knowledge.

3.1 Data Understanding

3.1.1 Data Inventory

A comprehensive inventory of data sources is a critical first step in understanding the landscape available for analysis and modeling. From the outset of this project, all collected data has been strategically organized into two main categories : (1) structured datasets intended for LLM fine tuning, mainly comprising formatted error–solution pairs ; and (2) unstructured documentation materials, primarily PDF guides and manuals, purposefully maintained for subsequent retrieval augmented generation (RAG) indexing.

This dual stream approach ensures that both supervised instruction following (via fine tuning) and dynamic document retrieval (via RAG) are grounded in the most relevant and up to date operational knowledge.

— **Server Logs :** Raw error logs from application servers, Java processes, and underlying databases (Oracle, etc.) provide detailed chronological records of system activity, failures, and anomalies within HR Access environments.

— **Jira Incident Tickets :** These tickets record a wide variety of errors including recurring, rare, and previously unseen incidents. They often include collaborative troubleshooting dialogues and, in some cases, practical solutions contributed by team members.

— **HR Access Documentation and Procedures :** Official documentation, procedural files, step by step guides, and best practice manuals authored by the team, spanning both internal and vendor provided sources.

— **Personal Notes and Communications :** Ad hoc notes made by operators and engineers, as well as archived chat transcripts (e.g., from Microsoft Teams) capturing informal exchanges and

knowledge sharing during troubleshooting.

— **GIFTS Database :** A specialized system for logging client reported errors, notable incidents, and related troubleshooting steps serving as an additional source of field level expertise and edge case documentation.

Overall, the data landscape is complex and scattered, with sources varying greatly in structure, granularity, and quality. This diversity poses significant challenges for aggregation, parsing, and integration, making the Data Understanding phase essential prior to any effective knowledge extraction or machine learning application.

3.1.2 Data Characteristics

Volume The JIRA ticket system stands out as the most resourceful single source of error and solution data ; it contains a substantial history of incidents and troubleshooting actions. In contrast, the number of standalone documents, manuals, and official documentation is relatively limited, and much procedural expertise remains uncaptured outside the ticketing system.

Variety Across sources, nearly all data is unstructured. Error logs are free text, tickets embed commentary and resolutions in an inconsistent prose format, and even many dedicated documentation files lack common structure or schema. Only a minority of documents contain highly structured procedural knowledge. Most content is application specific, with general HR Access product knowledge comparatively scarce and scattered.

Quality The dataset faces several critical quality challenges :

— **Missing Situations :** Many unique or team specific error cases are absent, especially those not previously documented, limiting representativity.

— **Inconsistency :** Solutions and troubleshooting notes can vary widely in detail, accuracy, and relevance from ticket to ticket or document to document.

— **Noise :** Irrelevant data such as verbose logs, unrelated comments, and legacy content complicates analysis and model training.

— **Outdated Information :** Some procedures and solutions reference outdated application versions, obsolete configurations, or deprecated system components.

3.1.3 Initial Analysis and Exploration

3.1.3.1 Overview

The available data can be aggregated from multiple sources ranging from critical server logs, incident tickets, and technical documentation to personal notes and chat transcripts. Depending on the source, collection may be manual, requiring careful extraction and verification, or automated, leveraging scripts and integrations to harvest data more efficiently.

3.1.3.2 Dataset and Heterogeneity

However, the process is anything but straightforward. The dataset is a mosaic scattered fragments collected from siloed systems, inconsistent repositories, and personal knowledge stores. Data comes in radically different formats, with no uniform structure or schema to unify content. With every new source included, the complexity grows : unformatted log entries, free text ticket comments, annotated PDFs, and legacy database dumps all converge to create a landscape that is rich in potential but daunting in its disorder.

3.1.3.3 Distribution

Within this chaotic terrain, certain patterns emerge. Some error types appear with alarming frequency, dominating logs and ticket histories. Others lurk on the margins rare, enigmatic, and difficult to diagnose. Documentation density varies wildly from topic to topic ; routine procedures may be well covered, while advanced troubleshooting paths exist only in isolated comments or scattered emails. Entities such as service names, error codes, and product modules cluster in unpredictable ways, reflecting both operational realities and historical quirks in how knowledge has been captured over the years.

This dramatic heterogeneity and uneven distribution illustrate the enormous challenge and the equally significant opportunity of transforming raw, dispersed data into structured, actionable intelligence for the team.

3.2 Data Preparation

The Data Preparation phase represents a critical bridge between chaotic raw data and structured assets ready for high-performance AI. Given the twin goals of our project, we deliberately constructed and prepared two distinct types of datasets :

— A formatted, instruction-based dataset derived primarily from Jira and logs for fine-tuning large language models (LLMs), enabling them to provide direct, context-rich, step-by-step solutions.

— An organized collection of unstructured PDF documents and guides serving as the knowledge base for the retrieval-augmented generation (RAG) system, ensuring dynamic, up-to-date referencing of procedural knowledge.

This dual dataset preparation ensures both accuracy (via supervised training) and depth/breadth (via real-time document retrieval), maximizing the power and flexibility of the AI assistant.

3.2.1 Data Collection

The collection of relevant data for this project was a truly formidable task, made vastly more challenging by the **fragmented** nature of operational knowledge throughout the organization. Information was scattered across isolated Jira tickets, buried deep in server logs, dispersed among informal chat histories, and locked within diverse documentation formats. No single tool or workflow could hope to bring order to this chaos alone.

Only by combining bespoke automated scraping solutions with meticulous, often painstaking manual collection were we able to capture this sprawling, **fragmented** landscape of knowledge. This foundational challenge set the stage for an equally rigorous phase of preprocessing and transformation, culminating in both a finely structured dataset for LLM fine-tuning and a maintainable archive of PDF resources to power RAG-based retrieval. The result is a carefully curated and modernized knowledge base, forged out of complexity by confronting head-on the realities of siloed and scattered enterprise data.

— **Automated Scraping from Jira :** A custom Python scraper was developed to systematically extract incident and error tickets from the Jira platform, covering all relevant cases logged over the past two years. This step enabled the retrieval of a large volume of 1678 historical troubleshooting records, capturing both the descriptions of incidents and the resolutions where available provided by the team.

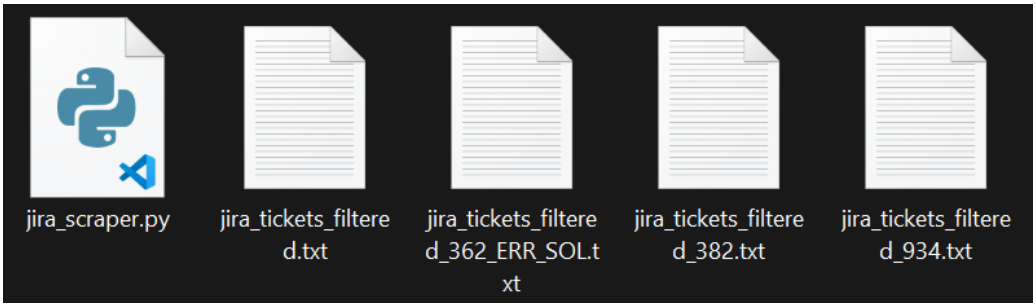


Figure 3.1 : Jira Scraper output files

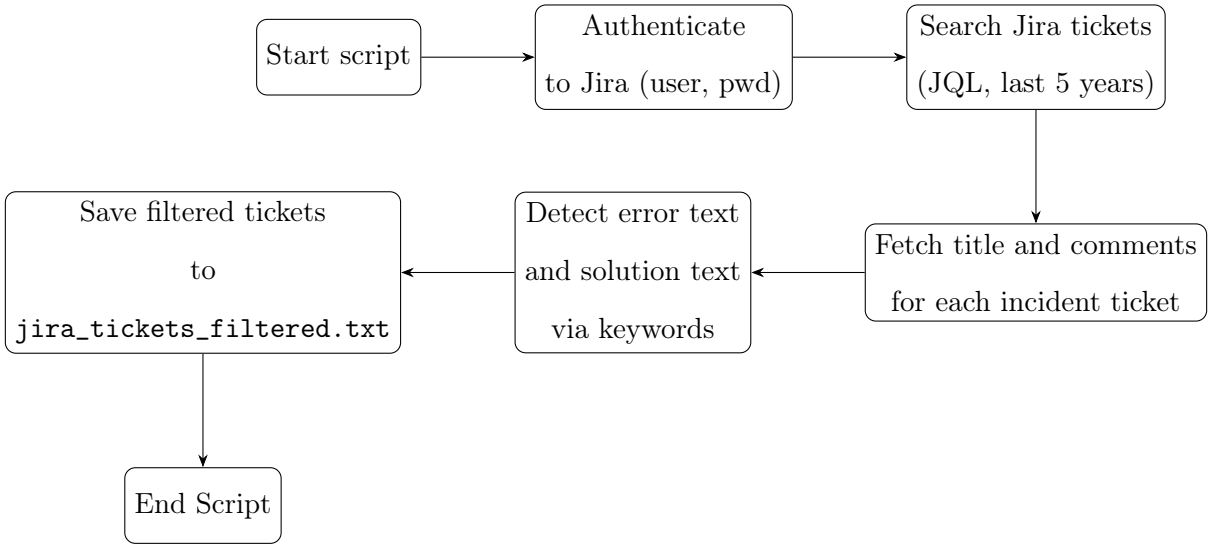


Figure 3.2 : Jira error–solution ticket extraction pipeline

— **Manual Search in Teams Channels :** Supplementary technical knowledge was sourced by manually reviewing chat histories and message archives in Microsoft Teams channels. This process aimed to recover valuable undocumented solutions and troubleshooting tips shared in real-time by engineers during incident resolution.

— **Manual Acquisition from GIFTS :** Data concerning client-reported issues and field-level solutions were manually exported from the GIFTS database. This required targeted searches for relevant records and additional validation to ensure completeness and contextual accuracy.

— **Manual Log Gathering :** Logs from application, database, and supporting system servers were manually collected, often as raw text exports or archived files. These logs provided granular details of error occurrences, system behaviors, and incident timelines, forming the backbone for various analytical and model-training efforts.

This hybrid collection strategy ensured comprehensive coverage of both formal and informal operational knowledge, maximizing the utility and representativeness of the final dataset.

3.2.2 Data Cleaning

The heterogeneity and raw nature of the collected data demanded a robust, multi-step cleaning process to ensure it would be both readable and valuable for LLM-based modeling and downstream analytics.

— **Scripted Cleaning for Scraped Data :** Automated scripts were employed as a first line of defense to sanitize the bulk of incident ticket data scraped from Jira and other digital sources. This included the removal of special characters, trimming excessive blank spaces, filtering out headers/footers, and eliminating irrelevant or uncontextual data fragments inadvertently

captured by the scraper. Regular expressions and standard text processing libraries played a core role in these cleaning operations.

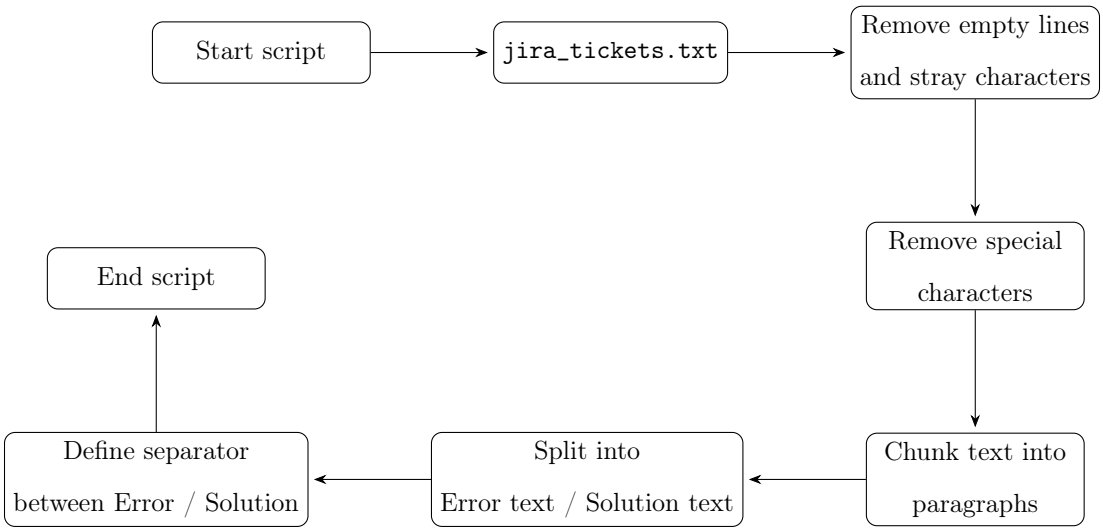


Figure 3.3 : Pipeline for preprocessing `jira_tickets.txt` into structured error–solution pairs

— **Manual Filtering and Refinement :** No automated process is foolproof especially with such scattered, inconsistently formatted data. Consequently, manual review was essential to catch erroneously included records and noise that slipped past initial scripted filters. This involved hand-inspecting for redundant or misplaced entries, removing partial or non-informative tickets, and making judgement calls about borderline-contributive snippets.

— **Data Enhancement and Human-Friendly Edits :** Critical records particularly those containing unique solutions or highly technical provenance were further refined through manual enhancement. Sentences were reworded for clarity, ambiguous references were resolved, and technical details were restated to be as accessible as possible for both humans and AI models. Where necessary, explanatory comments or context fields were added to boost the interpretability and instructional value of the data.

By combining large-scale automation with targeted human intervention, this cleaning pipeline produced a dataset that balances size, accuracy, and technical clarity laying the groundwork for robust and reliable model development.

3.2.3 Data Transformation

Following data cleaning, a critical transformation pipeline was required to convert raw incident data into a structure suitable for model training and retrieval tasks. The core of this process was the conversion of processed Jira tickets originally written as free-form reports and troubleshooting logs into clearly defined **error-solution pairs**.

Each ticket was carefully analyzed and split so that the description of the error, incident context, or technical question became the "instruction," while the corresponding solution, workaround, or procedural fix was designated as the "output." For cases with additional context or specific inputs (such as error codes or configuration details), these were mapped to the "input" field.

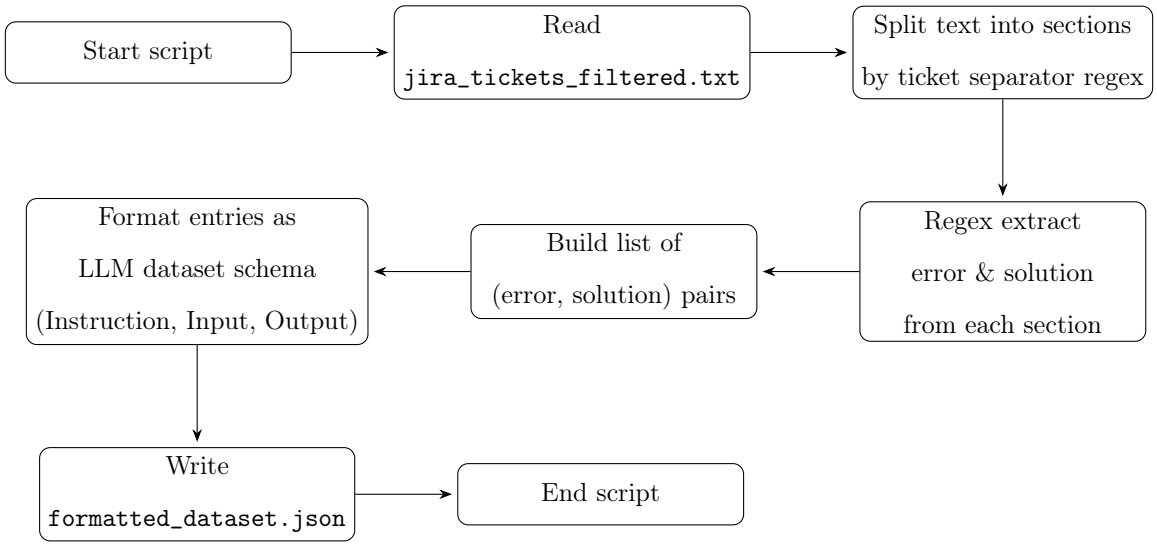


Figure 3.4 : Pipeline for extracting error–solution pairs and formatting them as an LLM training dataset

To enable downstream fine-tuning and benchmarking, the transformed data was formatted according to the widely-adopted Alpaca dataset schema [3].This approach supports supervised instruction following training for LLMs and consists of a flat collection of JSON objects, each with :

— `instruction` : The actionable error description or user query.

— `input` : Any supplementary context required to solve the problem (optional).

— `output` : The model-ready, cleaned solution or step-by-step fix.

This systematic transformation, a key element of data preprocessing, not only brings order and clarity to previously chaotic ticket histories, but also aligns enterprise data with modern best practices for LLM training maximizing the utility and accessibility of operational knowledge for both humans and intelligent assistants.

3.3 Conclusion

The Data Understanding phase revealed a complex landscape of fragmented enterprise data spread across multiple platforms and formats. A hybrid approach—automated scraping and manual searches of chats, logs, and legacy sources—was essential to gather useful knowledge. Rigorous preprocessing then cleaned, refined, and transformed this data to remove noise and ensure clarity for human and LLM use. Ultimately, this produced a structured fine-tuning dataset and an organized RAG archive,

combining accurate instruction-following with dynamic retrieval. Despite valuable content, especially within JIRA tickets, inconsistencies and gaps persist, guiding focused efforts toward consolidation and preparation.

Chapter 4

Modeling

Plan

1 **What is an LLM and Why Pretrained Models ?** 31

2 **Model Choice** 31

3 **Model Fine-Tuning** 35

4 **RAG : Retrieval Augmented Generation** 40

5 **RAFT : Retrieval Augmented Fine Tuning RAFT = RAG + FT** 44

6 **Conclusion** 46

Introduction

The Modeling phase builds on the intensive preparation efforts by transforming cleaned and structured operational knowledge into a practical AI assistant tailored to the team's needs. It combines a fine-tuned meta llama/Llama 3.1 8B Instruct model [4], adapted with LoRA[5][5] on Jira derived error–solution instruction pairs, with a RAG [6] pipeline that indexes key procedural PDFs and technical documents using Hybrid [7] sementic and BM25 [8] retrieval. Through this integration, supported by targeted prompt design and inference settings, the system aims to deliver fast, accurate, and context rich support grounded in up to date internal knowledge for the Production Operations RD team.

4.1 What is an LLM and Why Pretrained Models ?

A *large language model* (LLM) is a deep neural network with billions of parameters trained on massive text corpora to understand and generate human language [5], [6], [9], [10], [11], [12]. Modern LLMs are typically based on the transformer architecture and can perform many tasks such as answering questions, summarising documents, translating text, or writing code, all from the same underlying model [9], [10], [12], [13], [14], [15], [16]. In practice, training such a model from scratch would require enormous amounts of data, specialised infrastructure, and months of GPU time [9], [17], [18], [19]. For enterprise projects, it is therefore standard to start from a *pretrained* LLM : a model that has already learned general language patterns on large, mostly public datasets, and then adapt it to the target use case via fine-tuning or prompt engineering [17], [20], [21], [22], [23]. This approach drastically reduces cost and time-to-value while still allowing the injection of domain-specific knowledge (for example, Jira tickets and internal documentation) during a lighter adaptation phase [21], [24], [25], [26], [27].

4.2 Model Choice

4.2.1 Why Is Model Selection Important ?

Model selection is a high-stakes engineering decision, because a poorly chosen LLM can lead to system failures, excessive costs, and weak performance that undermines stakeholder trust [28], [29]. The model's architecture and pre-training quality determine how well it understands complex queries ; a mismatch increases the risk of harmful "hallucinations" that produce incorrect yet confident answers, whereas an appropriate model yields accurate, context-aware responses that improve efficiency and user satisfaction [30], [31]. At the same time, model size and compute footprint tightly constrain deployment,

forcing engineers to balance reasoning power against GPU memory, latency, and infrastructure budgets, often using quantization to fit models on available hardware without unacceptable loss of precision [32], [33], [34]. In this project, these trade-offs had to be aligned with Sopra Steria's concrete constraints—low-latency support, compatibility with internal GPUs, and RAG-oriented scalability—making model selection a strategic choice that directly conditions the assistant's long-term viability and business impact [2], [35].

4.2.2 Key Steps in Model Selection

4.2.2.1 Define Requirements

The process began with a rigorous requirements analysis, aligning concrete use cases and hardware limits to avoid later failures. The target assistant needed to handle complex IT scenarios (Linux diagnostics, SQL optimisation, HR Access workflows) while remaining deployable on mid-range GPUs such as NVIDIA T4 cards with 16,GB of VRAM, rather than expensive H100 clusters.[18], [19] Preference was given to instruction-tuned models that already follow prompts like "explain step by step," reducing fine-tuning effort and making the system more usable for non-experts. These needs were translated into measurable specifications—latency targets, acceptable hallucination rates, and seamless integration with Python tooling and the Hugging Face ecosystem—to guide all subsequent comparisons.[36], [37]

4.2.2.2 Research and Comparison

With requirements in place, a systematic survey compared leading open models from families such as Llama, Qwen, and Mistral, using public leaderboards and benchmarks to estimate their relative strengths in multilingual and technical tasks.[38], [39] Evaluation considered both architectural aspects (decoder-only transformers, context handling) and training data profiles, as well as reported scores on benchmarks like MMLU and HumanEval for reasoning and code generation. This research phase narrowed the field to a small set of candidates that were both performant and realistic to deploy under the project's hardware and cost constraints.

4.2.2.3 Manual Inference Testing

Because generic benchmarks do not fully capture enterprise-specific behaviour, each shortlisted model was then stress-tested on internal scenarios, including French queries, noisy log fragments, and HR Access–specific troubleshooting prompts. Using a controlled inference setup, the team measured latency, coherence, and robustness to ambiguous or partial inputs, revealing practical differences that raw scores had obscured. This manual probing surfaced strengths and weaknesses per model (for example, variance in step-by-step reasoning or domain vocabulary coverage), providing essential

evidence for an informed final choice, this procedure will be discussed in more detailed manner in the next chapter of Evaluation.

4.2.2.4 Evaluation Criteria

Final selection relied on a multi-criteria view that balanced accuracy, latency, memory footprint, and instruction-following quality rather than optimising a single metric. Accuracy focused on factual correctness for technical content and resistance to hallucinations, latency and VRAM usage ensured that 4-bit or similarly optimised deployments would remain responsive on available GPUs, and instruction adherence checked whether models reliably produced structured, concise procedures.[30], [32] Together, these criteria ensured that the chosen model was not only strong on paper but also realistically operable and maintainable within the project's infrastructure and business constraints.

4.2.3 Requirements and Constraints

Selecting the underlying large language model is a decisive factor for the assistant's overall performance, reliability, and long-term maintainability. The model must balance raw capability with practical constraints such as GPU memory, inference speed, operational cost, and ease of integration into an existing enterprise pipeline. In this project, additional requirements include strong instruction-following behaviour for assistant-like interactions, robust technical and troubleshooting reasoning, multilingual support (at least English/French), and compatibility with parameter-efficient fine-tuning methods (e.g. LoRA) and RAG-based retrieval over internal documentation.

4.2.4 Candidate Models

A preliminary survey focused on recent, high-performing models in the "small size, high performance" segment that are realistic to deploy on limited GPU resources. Based on public benchmarks, community evaluations, and initial hands-on experiments, three models emerged as primary candidates :

- `meta-llama/Llama-3.1-8B-Instruct`
- `Qwen/Qwen2.5-7B-Instruct`
- `mistralai/Mistral-7B-Instruct-v0.2`

All three provide competitive performance on a broad range of reasoning and instruction-following tasks while remaining small enough to be fine-tuned and served in an enterprise environment with constrained hardware.

4.2.5 Final Choice and Justification

After comparing benchmark performance, licensing, ecosystem maturity, and practical behaviour on project-specific test prompts, the project ultimately centred on `meta-llama/Llama-3.1-8B-Instruct`. This model offers a strong compromise between linguistic capability and resource footprint, with solid results on instruction-following and technical reasoning tasks in the 8B parameter class, while remaining feasible to fine-tune and serve on the available GPU infrastructure. Its open availability, support for efficient adaptation techniques such as LoRA, and good integration with existing tooling made it a natural backbone for building a specialised, domain-aware assistant, avoiding the prohibitive costs of training a large model from scratch.

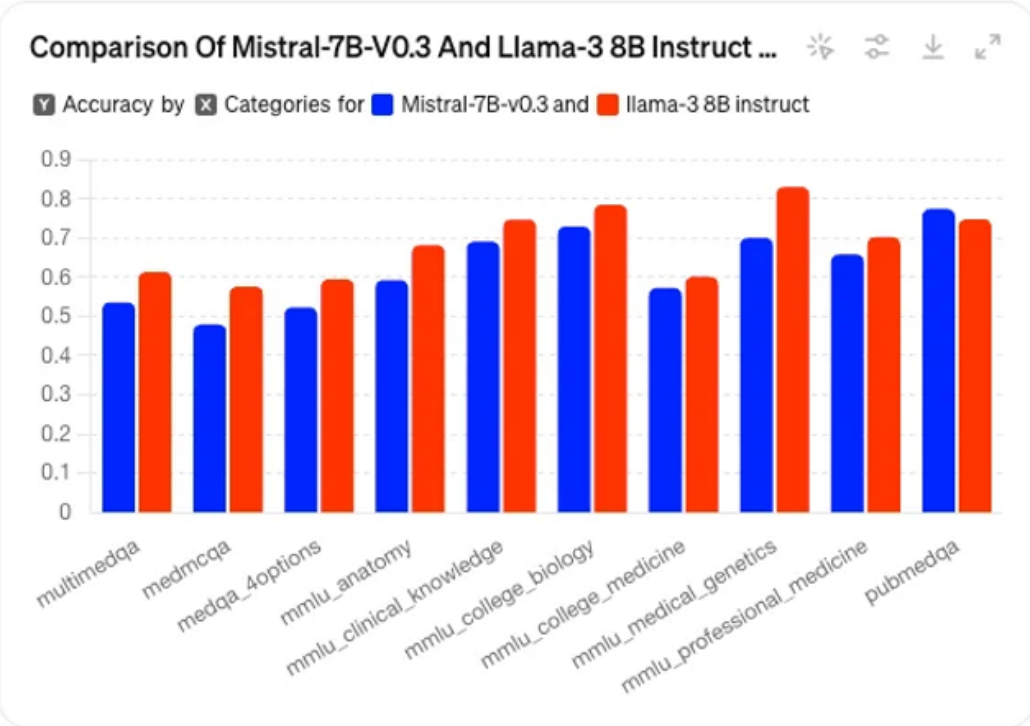


Figure 4.1 : LLama 3 8B - Mistral 0.3 7B Benchmark

The final decision was made through manual inference testing on a representative set of HR Access and infrastructure related technical scenarios. These experiments evaluated not only correctness, but also clarity of explanation, robustness to noisy queries, and latency under realistic hardware constraints. `meta llama/Llama 3.1 8B Instruct`[4] was ultimately selected as the base model. It provided a strong combination of instruction following quality, stable behavior on domain specific technical questions, and efficient inference when quantized and adapted with LoRA[5]. This model therefore offered the best trade off between accuracy, resource usage, and extensibility for subsequent fine tuning and RAG integration within the constraints of the project[40].

4.3 Model Fine-Tuning

The training phase is where the theoretical design choices of the modeling step turn into a concrete, running system. In practice, this phase determines how well the selected LLM internalizes the enterprise-specific knowledge curated in the previous chapters and whether it can do so within the tight hardware and time constraints of a production environment. Far from being a simple "fit" operation, training the model for Sopra HR's Production Operations R&D context required careful prompt engineering, parameter-efficient fine-tuning, aggressive optimization of GPU usage, and rigorous experiment tracking. Every configuration decision (sequence length, precision mode, LoRA[5]rank, optimizer, batch size) directly impacts both model quality and system stability, making this phase a high-responsibility engineering effort.

4.3.1 Prompt Engineering for Supervised Training

Before any gradient is computed, the way training data is presented to the model must be precisely engineered. In this project, raw Jira-derived error–solution pairs were transformed into a structured instruction format that the LLM could consistently learn from. Each training example is wrapped in a prompt template that :

- Defines the model's role as a highly skilled IT expert specialised in system administration, Linux, SQL databases, server management, and HRACCESS environments.
- Separates the `Instruction` (what the model is expected to do), the `Question` (problem description), and the `Response` (ground-truth solution).
- Encourages step-by-step reasoning by explicitly asking the model to think carefully before answering.

Programmatically, a formatting function converts each JSON record `{Question, Response}` into a single training string stored in the `text` field, with an end-of-sequence token appended for the decoder. Conceptually, this is similar to training a pilot not only on "takeoff" and "landing" outcomes, but on full cockpit procedures : the model internalizes not just the answer, but the structure and tone of expert technical reasoning. Poor or inconsistent prompts would otherwise introduce behavioural noise, even with high-quality underlying data.

```
train_prompt_style = """Below is an instruction that describes a task, paired with an input that provides further context.
Write a response that appropriately completes the request.
Before answering, think carefully about the question and create a step-by-step chain of thoughts to ensure a logical and accurate response.

### Instruction:
You are a highly skilled IT expert specializing in system administration, Linux, SQL databases, server management, and HRACCESS environments.
Your role is to provide precise, technically accurate, and well-structured solutions to complex IT-related queries.

### Question:
{}

### Response:
{}"""
```

Figure 4.2 : Training Prompt Design

4.3.2 Efficient Fine-Tuning with Unsloth and LoRA

Given that the base model `meta-llama/Llama-3.1-8B-Instruct` contains billions of parameters, naïve full-precision fine-tuning would be prohibitively expensive and often impossible on budget enterprise GPUs such as NVIDIA Tesla T4 cards[19]. To overcome these constraints, the project relies on the Unsloth framework[41], an open-source library that accelerates and memory-optimises LLM fine-tuning up to 30x by combining quantization, parameter-efficient methods such as LoRA[5], and other training optimisations into a unified workflow [41], which integrates several advanced optimisation techniques into a cohesive pipeline :

— **Low-precision loading and quantization :** The base model is loaded in 4-bit quantised form (`load_in_4bit = True`), dramatically shrinking its memory footprint so that an 8B-parameter model can run on GPUs with limited VRAM.

— **Parameter-efficient fine-tuning (LoRA) :** Instead of updating all weights, small trainable LoRA[5]adapters are injected into key transformer components (attention projections `q_proj`, `k_proj`, `v_proj`, `o_proj` and feed-forward layers `gate_proj`, `up_proj`, `down_proj`). Only these low-rank matrices (e.g. rank $r = 16$) are trained, while the original parameters remain frozen.

— **Gradient checkpointing :** Intermediate activations are recomputed during backpropagation instead of stored, trading some computation for a substantial reduction in VRAM and enabling longer sequence lengths without out-of-memory errors.

— **Mixed-precision training :** The configuration automatically selects FP16 or BF16 based on GPU support, balancing numerical stability and speed. Combined with 4-bit base weights, this yields a highly efficient multi-precision stack.

— **Dataset flexibility :** Unsloth[41]readily ingests instruction-style datasets such as the Jira error–solution pairs used here, minimizing glue code and letting engineering effort focus on data quality rather than boilerplate.

In effect, Unsloth[41]acts as a high-performance “fine-tuning engine” that hides much of the complexity of coordinating quantisation libraries,custom kernels, and training loops, while still exposing expert-level controls when needed.

```python
# Apply LoRA (Low-Rank Adaptation) fine-tuning to the model
model_lora = FastLanguageModel.get_peft_model(
    model,
    r=16,  # LoRA rank: Determines the size of the trainable adapters (higher = more parameters, lower = more efficiency)
    target_modules=[  # List of transformer layers where LoRA adapters will be applied
        "q_proj",   # Query projection in the self-attention mechanism
        "k_proj",   # Key projection in the self-attention mechanism
        "v_proj",   # Value projection in the self-attention mechanism
        "o_proj",   # Output projection from the attention layer
        "gate_proj",  # Used in feed-forward layers (MLP)
        "up_proj",    # Part of the transformer's feed-forward network (FFN)
        "down_proj",  # Another part of the transformer's FFN
    ],
    lora_alpha=16,  # Scaling factor for LoRA updates (higher values allow more influence from LoRA layers)
    lora_dropout=0,  # Dropout rate for LoRA layers (0 means no dropout, full retention of information)
    bias="none",  # Specifies whether LoRA layers should learn bias terms (setting to "none" saves memory)
    use_gradient_checkpointing="unsloth",  # Saves memory by recomputing activations instead of storing them (recommended for long-context fine-tuning)
    random_state=3407,  # Sets a seed for reproducibility, ensuring the same fine-tuning behavior across runs
    use_rslora=False,  # Whether to use Rank-Stabilized LoRA (disabled here, meaning fixed-rank LoRA is used)
    loftq_config=None,  # Low-bit Fine-Tuning Quantization (LoFTQ) is disabled in this configuration
)
```

Figure 4.3 : LoRA Algorithm and Used Parametres with Respective Description

4.3.3 Supervised Fine-Tuning with `SFTTrainer`

On top of this optimisation stack, supervised fine-tuning is performed using the `SFTTrainer`[42] from the Hugging Face TRL library[43], as integrated by Unsloth[41]. The pipeline begins by organising the domain-specific dataset (formatted Jira question–response pairs) into a dedicated `"text"` field, where each entry is a self-contained prompt including instruction, question, and reference answer.

During training, the model processes mini-batches rather than the entire dataset at once, respecting GPU memory limits. For each batch, the model :

— Receives an input sequence (full instruction–question prompt),

— Produces a predicted continuation (its answer),

— Compares this prediction to the ground-truth response via a token-level loss (e.g. cross-entropy),

— Updates only the LoRA[5]adapter weights through backpropagation.

Key features and settings include :

— **Supervised fine-tuning :** Classic input–output learning on labelled examples, improving instruction-followin and domain-specific behaviour.

— **Optimiser and precision :** Memory-efficient optimisers (e.g. AdamW in 8-bit) plus mixed precision to exploit GPU capabilities while maintaining stability.

— **Flexible controls :** Exposed hyperparameters for batch size, gradient accumulation, learning rate, maximum sequence length, and training steps allow both quick debugging runs and longer production fine-tunes.

— **LoRA and quantisation awareness :** The trainer knows how to update only LoRA[5]adapters on top of a quantised base model, enabling high-quality customisation on modest GPUs.

— **Logging and checkpointing :** Automatic metric logging, checkpoint saving, and fixed random seeds support reproducibility and safe resumption of runs.

Because the base model already encodes broad language knowledge, fine-tuning does not “teach it language from scratch” ; it nudges an existing high-capacity system toward the specific distribution of Sopra HR tickets and procedures.

```
# Initialize the fine-tuning trainer — Imported using from trl import SFTTrainer
trainer = SFTTrainer(
    model=model_lora,  # The model to be fine-tuned
    tokenizer=tokenizer,  # Tokenizer to process text inputs
    train_dataset=dataset_finetune,  # Dataset used for training
    dataset_text_field="text",  # Specifies which field in the dataset contains training text
    max_seq_length=max_seq_length,  # Defines the maximum sequence length for inputs
    dataset_num_proc=2,  # Uses 2 CPU threads to speed up data preprocessing

    # Define training arguments
    args=TrainingArguments(
        per_device_train_batch_size=2,  # Number of examples processed per device (GPU) at a time
        gradient_accumulation_steps=4,  # Accumulate gradients over 4 steps before updating weights
        num_train_epochs=1, # Full fine-tuning run
        warmup_steps=5,  # Gradually increases learning rate for the first 5 steps
        max_steps=60,  # Limits training to 60 steps (useful for debugging; increase for full fine-tuning)
        learning_rate=2e-4,  # Learning rate for weight updates (tuned for LoRA fine-tuning)
        fp16=not is_bfloat16_supported(),  # Use FP16 (if BF16 is not supported) to speed up training
        bf16=is_bfloat16_supported(),  # Use BF16 if supported (better numerical stability on newer GPUs)
        logging_steps=10,  # Logs training progress every 10 steps
        optim="adamw_8bit",  # Uses memory-efficient AdamW optimizer in 8-bit mode
        weight_decay=0.01,  # Regularization to prevent overfitting
        lr_scheduler_type="linear",  # Uses a linear learning rate schedule
        seed=3407,  # Sets a fixed seed for reproducibility
        output_dir="outputs",# Directory where fine-tuned model checkpoints will be saved
    ),
)
```

Figure 4.4 : SFTTrainer Algorithm and Used Parametres with Respective Description

4.3.4 Experiment Tracking with Weights & Biases

Weights & Biases (W&B) is used as the experiment tracking and model monitoring platform for this project, providing centralized logging of metrics, configurations, and artifacts across fine-tuning runs [44]. It provides :

— **Real-time monitoring :** Live dashboards for training loss, validation metrics, learning rate schedules, and GPU utilisation, making it easier to detect divergence, overfitting, or resource bottlenecks.

— **Run comparison and reproducibility :** Each run is logged with configuration, code version, and dataset references, allowing side-by-side comparison of experiments and precise reproduction of promising configurations.

Integrating W&B turns training into a structured, data-driven process and underpins future retraining cycles with clear audit trails, more on the metrics and graphs in the next chapter of Evaluation.

Figure 4.5 : W&B Graphs

4.3.5 Optimisation, Quantisation, and Practical Impact

The deliberate combination of Unsloth[41], LoRA[5], quantisation, mixed precision, and disciplined experiment tracking transforms this project from a proof of concept into an operationally realistic system. The main impacts are :

— **Speed and cost :** 4-bit loading plus LoRA[5]fine-tuning allows training on a single GPU in 30 minutes rather than requiring multi-GPU clusters over days, directly reducing infrastructure and cloud costs and enabling rapid iteration.

— **Memory efficiency :** Training only a tiny fraction of parameters, together with gradient checkpointing and mixed precision, slashes VRAM requirements so that an 8B model can be fine-tuned on hardware tiers usually reserved for classical ML workloads.

— **Domain adaptation and modularity :** Because only LoRA[5]adapters are updated, multiple domain-specialised variants can be created and swapped cheaply on top of the same base model, simplifying deployment, versioning, and rollback.

— **Engineering discipline :** Using Unsloth[41] and `SFTTrainer` codifies best practices for optimiser configuration, learning-rate scheduling, precision handling, and checkpoint layout into a repeatable training recipe, rather than ad-hoc scripts.

— **Software–hardware co-design :** Batch size, sequence length, precision, and quantisation settings are tuned explicitly against hardware limits, treating the GPU as a constrained device whose memory, compute, and throughput characteristics shape the training design.

Overall, the training phase weaves together prompt design, parameter-efficient fine-tuning, low-level optimisation, and rigorous tracking to inject Sopra HR's operational expertise into a state-of-the-art LLM, while remaining within strict hardware and budget limits. This turns a general-purpose model

into a reliable, domain-aware assistant that is both technically robust and economically sustainable for enterprise deployment.

4.3.6 LLM Fine-Tuning architecture overview

The following figure presents the end-to-end LLM fine-tuning pipeline used in this project, from data preparation to deployment of the adapted model. It highlights how raw examples are transformed into instruction-style prompts, how the base Llama-3.1-8B-Instruct model is adapted with LoRA via Unsloth, and how training, experiment tracking, and model publishing to the Hugging Face Hub are orchestrated in a single workflow :

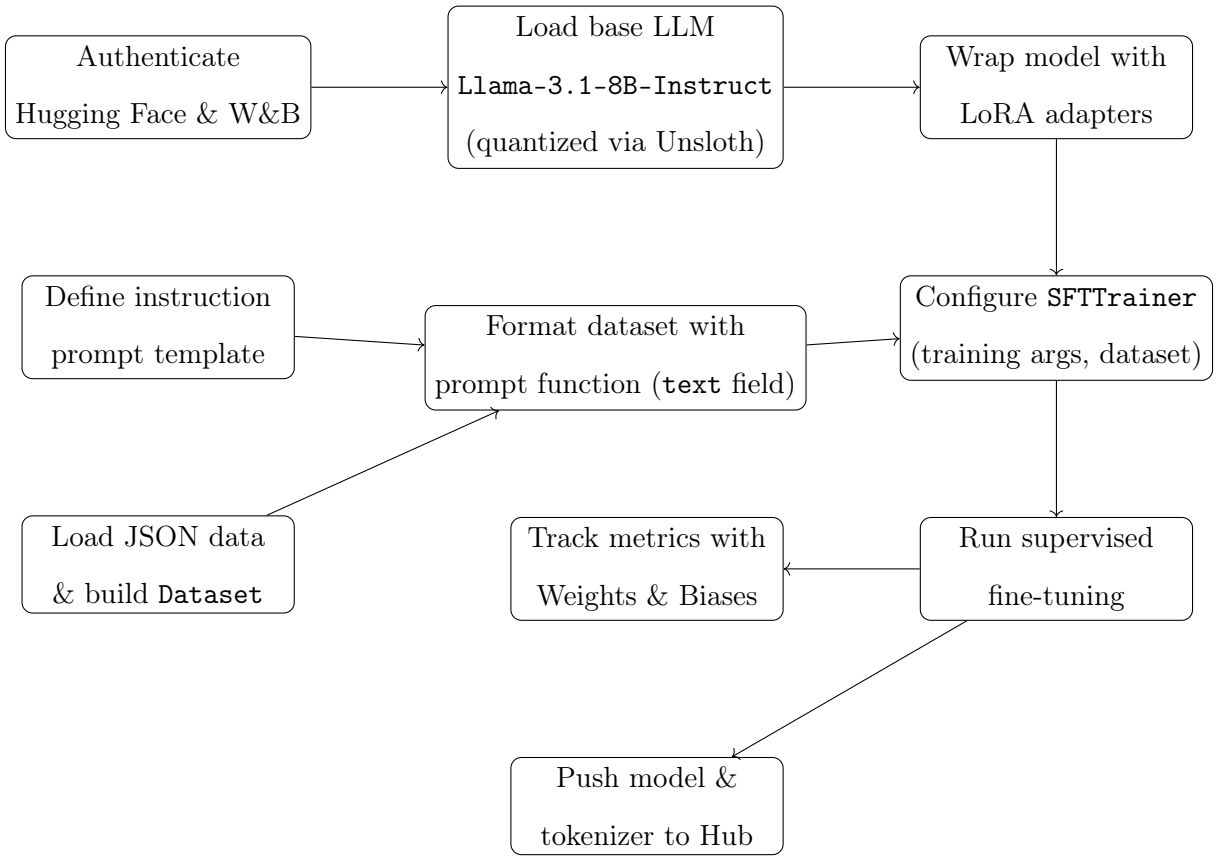


Figure 4.6 : Pipeline for Model Fine Tuning

4.4 RAG : Retrieval Augmented Generation

Retrieval Augmented Generation (RAG) is an AI architecture that couples information retrieval over external sources (documents, databases, knowledge bases) with large language models.[6], [45], [46], [47], [48], [49] Instead of answering purely from what was memorised during pre-training, the system first retrieves relevant pieces of external knowledge and then injects them, along with the user's question, into the model's context so that the final answer is explicitly grounded in up-to-date, authoritative data.[6], [46], [48], [49], [50] In practice, a RAG pipeline performs document ingestion

and chunking, encodes chunks and queries into dense vectors, retrieves the most relevant pieces via a vector or hybrid (dense + BM25) search engine, and uses a prompt template to feed this context into the LLM, which reduces hallucinations, enables querying private or highly specialised content without retraining the base model, and keeps the system current simply by refreshing the indexed knowledge base.[6], [7], [8], [30], [46], [48], [50], [51]

4.4.1 Importance of RAG for Enterprise LLMs

For engineering teams, RAG is the backbone that makes LLMs production-grade in complex environments such as HRIS, payroll, or production operations.[6], [45], [46], [47], [48], [49], [50] Instead of endlessly re-fine-tuning models whenever a process changes, teams design a retrieval layer : ingestion pipelines for PDFs and tickets, chunking strategies tuned to document structure, embedding jobs, and indexing workflows that can be scheduled, monitored, and rolled back like any other data pipeline, shifting the problem from "retrain the model" to "update the index".[7], [8], [46], [48], [50], [51] RAG also brings observability and control that pure fine-tuning cannot : engineers can inspect which chunks were retrieved, track similarity scores, A/B-test embedding models, or swap vector stores without touching the LLM, while security and architecture teams enforce data residency, scale retrieval independently of generation, and monitor latency at each hop.[46], [48], [49], [50] From a systems-engineering perspective, this enables domain-specialised copilots that respect strict functional and non-functional requirements : retrieval can be restricted to specific knowledge spaces, hybrid search can be tuned to surface rare technical edge cases, and rerankers can prioritise officially validated procedures over informal notes, turning messy enterprise knowledge (Confluence pages, Jira tickets, legacy PDFs) into an indexed, queryable substrate exposed through auditable, governance-aligned LLM interfaces.[7], [8], [30], [46], [48], [50]

4.4.2 RAG Components and Pipeline

4.4.2.1 Document Ingestion and Chunking

The first step is ingestion : importing raw enterprise data (PDFs, manuals, runbooks, tickets) and normalising it into a text format that downstream components can process (e.g. `Document` objects rather than binary PDFs). Long documents are then split into smaller, coherent chunks (nodes), typically aligned with sentences or paragraphs. This chunking controls input size for embeddings and LLMs, improves retrieval quality by aiming for "one main idea per chunk", and preserves coherence by using overlapping windows, so that important context is not cut across boundaries.

4.4.2.2 Embeddings and Vector Indexing

Each chunk, and later each user query, is converted into a numerical vector called an *embedding*, placing it in a high-dimensional space where semantic similarity corresponds to geometric proximity. A pre-trained embedding model (e.g. a sentence-embedding model from Hugging Face) reads the text and outputs a fixed-length vector, and a vector index (or vector database) stores these embeddings together with metadata (source document, page, position). This index enables fast approximate nearest-neighbour search instead of linear scans and preserves traceability by keeping links back to the original content.

4.4.2.3 Semantic and Lexical Retrieval

At query time, RAG performs retrieval to find the most relevant chunks for the user's question. Semantic (dense) retrieval computes an embedding for the query and searches the vector index for the closest chunk vectors, capturing meaning even when the wording differs (e.g. paraphrases in French or loosely phrased questions). Lexical retrieval, typically implemented with BM25, complements this by ranking chunks based on exact token matches, term frequency, and document length, which is crucial for catching precise items such as error codes, IDs, field labels, or HRaccess transaction names that embeddings might smooth away.

4.4.2.4 Hybrid Search and Context Construction

Hybrid search fuses semantic and lexical signals into a single context set that will be passed to the LLM. In practice, the system runs dense and BM25 retrieval separately, collects the returned node texts into two sets, takes their union to deduplicate overlaps, and optionally filters by similarity thresholds or relevance scores. The resulting combined context contains both conceptually similar passages and lexically precise matches, providing a richer and more reliable evidence base for generation, which directly improves recall (fewer missed relevant chunks) and precision (fewer off-topic passages).

4.4.2.5 Prompt Augmentation and LLM Generation

The final steps are prompt augmentation and LLM inference. The user query and the retrieved context are wrapped into a carefully designed prompt template that clearly specifies the objective ("answer based only on the context"), presents the context, and then restates the query before an answer section. If no relevant chunks are found, a fallback instruction-style prompt without context is used. The augmented prompt is tokenised and fed to the LLM with conservative decoding parameters (e.g. low temperature, small `top_k`, limited `max_new_tokens`) to favour deterministic, concise answers. In this way, the model becomes a generator grounded in the retrieved enterprise knowledge, rather than a free-form text predictor, yielding responses that are both useful and auditable for Sopra HR's

Production Operations R&D environment.

4.4.3 RAG Architecture Overview

The first figure depicts the retrieval and text-processing pipeline that ingests enterprise documents, chunks them, embeds them, and performs vector and BM25 retrieval to build a grounded RAG context for each query.[6], [7], [8], [46], [48], [50]

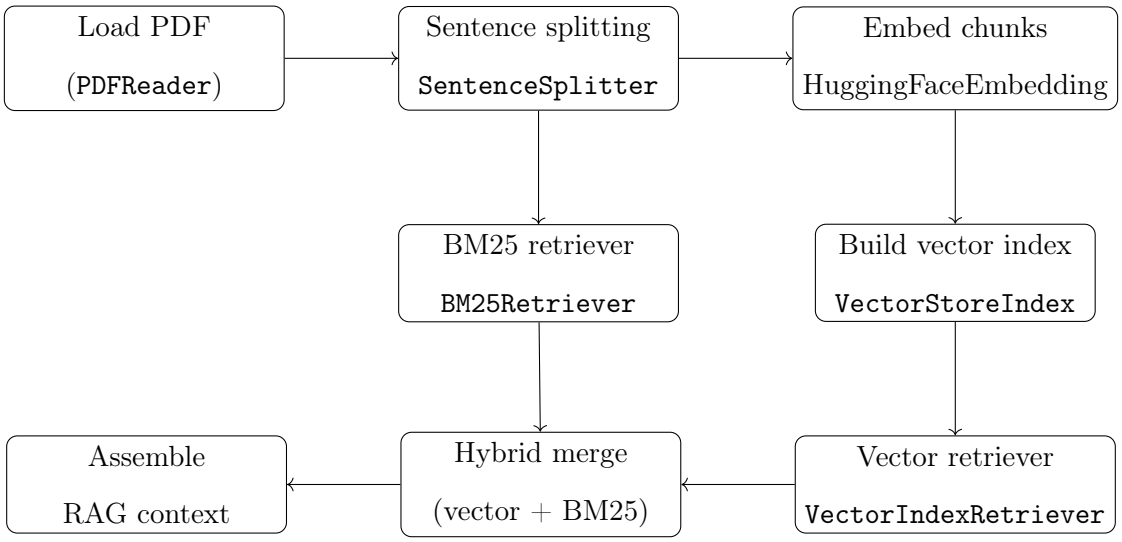


Figure 4.7 : Pipeline for Hybrid Retrieval

The second figure shows the LLM inference pipeline, where the system loads the fine-tuned, quantized model, selects an appropriate prompt (RAG or normal), tokenizes the input, and generates the final answer conditioned on the retrieved context and user query.[4], [41], [43]

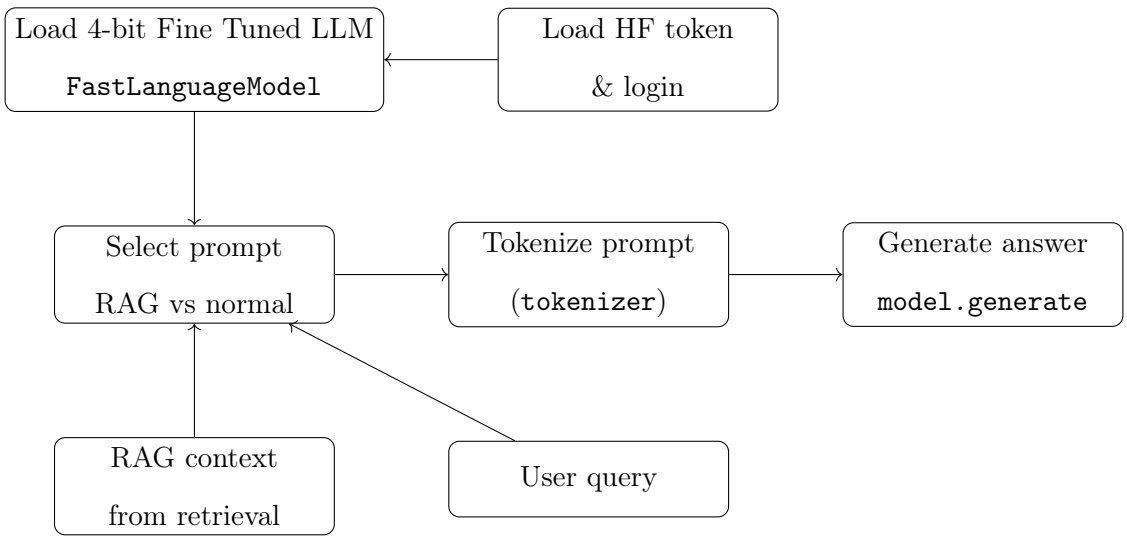


Figure 4.8 : Pipeline for RAG and LLM inference

4.5 RAFT : Retrieval Augmented Fine Tuning RAFT = RAG+FT

RAFT (Retrieval Augmented Fine Tuning) represents the state of the art in domain adaptation for enterprise LLM systems.[52], [53], [54], [55] RAFT is, at its core, the sum of Retrieval Augmented Generation (RAG) and targeted Fine Tuning (FT) on custom, domain specific data : RAFT = RAG + Fine Tuning (FT).[52], [53], [55], [56], [57], [58], [59]

End Result : RAFT delivers a language model that not only "reads" external documents on the fly via a RAG pipeline, but is also explicitly trained (fine-tuned) to interpret and reason using those retrieved documents, achieving higher accuracy, reasoning depth, and robustness than using RAG or fine tuning alone.[52], [53], [54], [55], [58], [60]

4.5.1 RAFT : State of the Art

- **Best of Both Worlds :** fine-tuned models learn in context reasoning skills using domain specific tasks, while RAG ensures the model is always grounded in the freshest knowledge. RAFT blends these strengths allowing rapid updates (just add documents) and precise, optimized answer quality on internal workflows.
- **Engineering Excellence :** RAFT systems those that blend retrieval augmented generation with fine tuning are a clear example of smart engineering. By training models not only to pull in relevant, up to date information from outside sources, but also to reason deeply about that information, RAFT creates AI solutions that are accurate and adaptable in complex domains. With carefully built pipelines and feedback loops, these systems handle new information smoothly, reduce errors, and scale up for real world needs. RAFT's modular design shows how the best AI relies on both dynamic knowledge and careful tuning to meet today's demanding tasks.
- **Algorithmic Tailoring :** As part of this work, I had to design and orchestrate several algorithms so that they work together as a coherent system rather than isolated components, Concretely, this involves :
  - carefully balancing retrieval precision and recall (e.g. tuning the oracle/distractor ratio used during training so the model learns to pick the truly relevant chunk among many candidates),
  - ensuring the model learns to ignore irrelevant or noisy context instead of overfitting to anything retrieved,
  - integrating knowledge indexes, dense and sparse retrievers, rerankers, and in-context reasoning logic into a seamless training and inference loop,
  - engineering a robust hybrid search layer (like the custom vector+BM25 fusion implemented

in this project) so that the fine-tuned model sees high quality, de-duplicated context that matches both the retrieval algorithms and the available enterprise data.

— **Versatile Benefits :** RAFT models outperform classic fine-tuned or RAG only models on :

  — *Accuracy :* Superior question answering on proprietary documents.

  — *Resilience :* Can robustly handle distractors and retrieval "mistakes."

  — *Efficiency :* Lower data, compute, and maintenance costs compared to static re training.

In summary, RAFT represents the culmination of both data curation and engineering innovation : it is the result of tailoring model training and retrieval logic to perfectly fit an organization's unique data landscape delivering answers that are as trustworthy and up to date as a human expert with instant search superpowers.

4.5.2 RAFT Architecture Overview

The following diagram provides an end-to-end overview of the RAFT architecture, from supervised fine-tuning to retrieval-augmented inference.[52], [53], [54], [55] At training time, the base LLM is adapted with supervised fine-tuning on instruction data that explicitly includes retrieved documents as context, so the model learns to read, interpret, and reason over external knowledge rather than relying only on its pre-training.[52], [53], [59], [60]

At inference time, a user query first passes through the RAG retrieval layer, which ingests the question, searches the enterprise indexes (vector and/or hybrid retrieval), and assembles a compact context bundle that is injected into a RAFT-style prompt alongside the query.[6], [46], [48], [50] The fine-tuned RAFT model then generates an answer conditioned on both the query and the retrieved context, yielding responses that are more accurate, domain-aware, and robust than those produced by either plain RAG or plain fine-tuning alone.[52], [53], [55], [58]

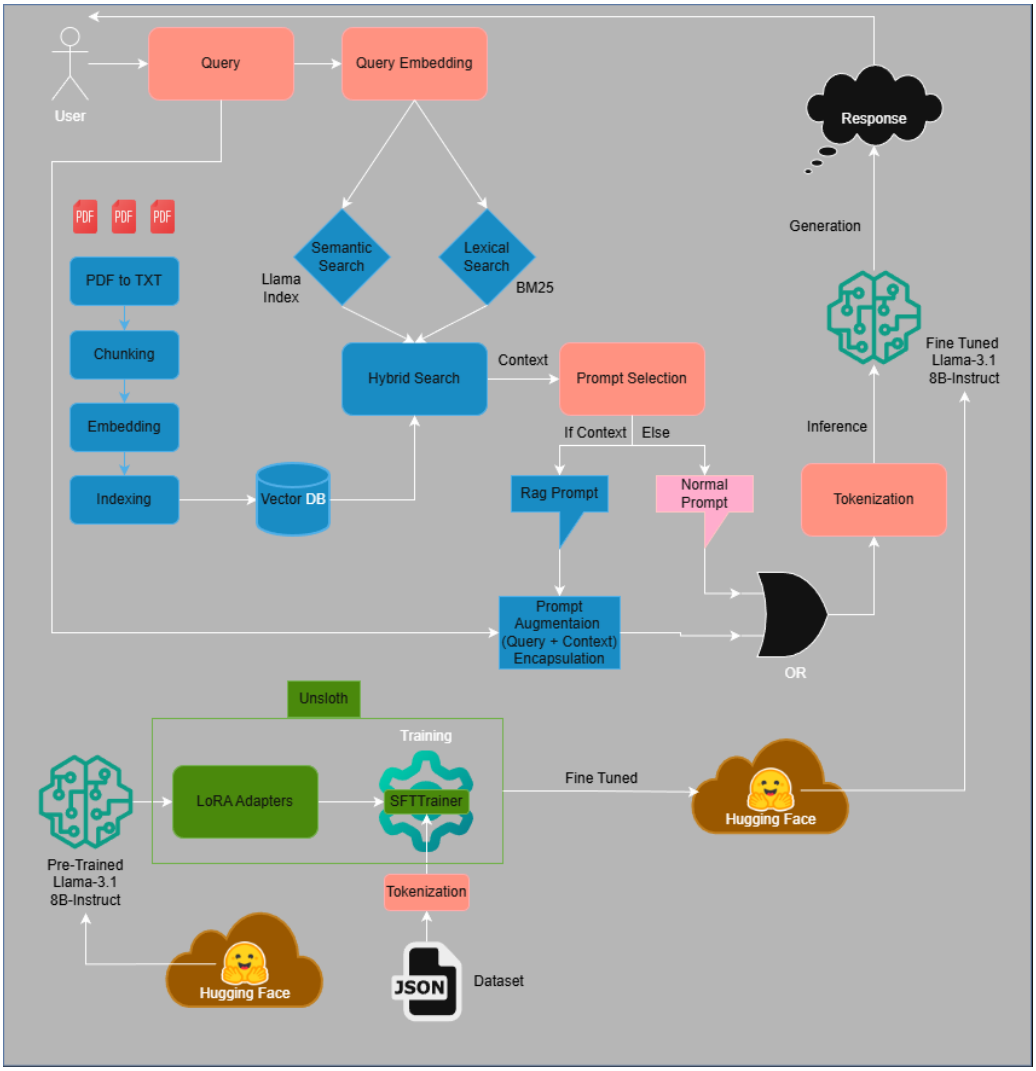


Figure 4.9 : Global Architecture

4.6 Conclusion

This Modeling phase implemented a RAFT approach, combining a RAG backbone with supervised fine-tuning of the base LLM so that enterprise documents are ingested, chunked, embedded, indexed, and queried through a hybrid (vector + BM25) search layer while the model is explicitly trained to interpret retrieved chunks and follow Sopra HR / HRACCESS procedures, yielding deterministic, operationally useful answers within a coherent architecture where data pipelines, retrieval algorithms, and the fine-tuned LLM form an integrated, extensible foundation for subsequent Evaluation and Deployment phases.[7], [8], [21], [23], [46], [50], [52], [53]

Chapter 5

Evaluation and Deployement

Plan

1 Evaluation Objectives and Methodology 48

2 Test Environment 49

3 Pre-Trained LLM Evaluation for Model Selection 51

4 Fine-Tuned LLM Evaluation 52

5 RAG : Context Retrieval Evaluation 55

6 RAFT : Full Project Outcome Evaluation 56

7 Ready To Deploy Model 57

Introduction

This chapter closes the CRISP-DM cycle by assessing whether the RAFT solution a fine-tuned LLM coupled with a hybrid RAG pipeline meets the initial objectives and is ready for realistic operational use, with attention to robustness, failure modes, and production constraints rather than raw accuracy alone.[6], [46], [50], [52], [53] During evaluation, issues such as incomplete or noisy retrieval, over-confident incorrect answers, and edge cases involving ambiguous queries or rare procedures led to iterative adjustments of chunking, similarity thresholds, prompts, and training data to improve grounding and reduce hallucinations.[7], [8], [21], [30], [48], [51] The final artefact is a "ready-to-deploy" package : LoRA-based fine-tuned weights, tokenizer, and configuration published on a model hosting platform along with inference and RAG pipeline code, so that future deployment phases can focus on integration concerns such as security, monitoring, and user interfaces rather than re-engineering the core model.[4], [37], [41], [43]

5.1 Evaluation Objectives and Methodology

The primary goal of the evaluation phase is to verify whether the developed solution a fine-tuned large language model combined with a hybrid retrieval-augmented generation (RAG) pipeline actually delivers useful, reliable, and robust assistance for the targeted operational context. Beyond checking that the model "works" on a few examples, the evaluation aims to answer four key questions :

— Are the answers technically correct and understandable (quality) ?

— Is the system stable and predictable across different runs and queries (reliability) ?

— Does it behave sensibly in the presence of noisy, incomplete, or ambiguous inputs (robustness) ?

— Does it provide a real productivity gain for typical users compared to the existing, purely manual process (usefulness) ?

To address these questions, a simple but structured evaluation protocol was defined. First, a dedicated test set was constructed from tickets, incidents and scenarios that were *not* used during fine-tuning, in order to approximate real "unseen" situations. These scenarios cover a representative variety of difficulties : common problems, rare edge cases, different levels of detail, and both well-structured and noisy inputs. For each test case, the system is executed in two modes when relevant : (i) the fine-tuned LLM alone, and (ii) the full LLM + RAG stack with hybrid retrieval, so that the impact of external context can be explicitly measured.

Evaluation is then carried out using a manual scoring grid applied by a human evaluator (or several, when available). Each answer is graded along several dimensions, for example :

— **Accuracy** : Is the proposed solution factually correct and technically sound for the given problem ?

— **Usefulness** : Would this answer be genuinely helpful for an engineer or consultant trying to resolve the issue ?

— **Grounding / hallucination** : Does the answer stay within what can be justified by the internal documentation and expected domain knowledge, or does it invent steps, commands or configurations without basis ?

— **Response time** : Is the latency (including retrieval and generation) acceptable for interactive use ?

From these scores, simple metrics are derived, such as percentage of "acceptable" or "good" answers, hallucination rate (fraction of answers judged misleading or unsupported), and average response time. Qualitative observations (typical failure modes, recurring strengths, impact of RAG compared to the base LLM) are recorded alongside numeric metrics. This mixed quantitative–qualitative methodology provides a balanced view : numbers to assess whether the system meets the defined targets, and detailed commentary to understand *why* it succeeds or fails and how it can be improved in future iterations.

5.2 Test Environment

All experiments, including fine-tuning, inference, and RAG pipeline evaluation, were conducted within the **Kaggle** computational environment. Kaggle[61] is a cloud-based platform for data science and machine learning that provides free access to containerized Jupyter notebook environments equipped with GPUs, CPUs, and preinstalled scientific libraries. It enables researchers and engineers to prototype, train, and evaluate models in a reproducible and shareable manner through hosted runtime sessions and integrated dataset management.

This platform provided a stable, containerized setup with GPU acceleration suitable for both development and controlled testing phases, including LLM fine-tuning, hybrid retrieval evaluation, and performance benchmarking.

5.2.1 Overview of the Kaggle Environment

Kaggle offers on-demand Jupyter-based environments[61] with integrated GPU and CPU resources, enabling reproducible machine learning experiments. The system runs containerized Linux[62] instances, preloaded with essential Python[63] libraries, notebook integration[64], and direct access to cloud storage for datasets, model artifacts, and evaluation outputs.

This setup was chosen for its balance of accessibility, reproducibility, and sufficient computing

power to handle fine-tuning of 8B-scale models under quantized and parameter-efficient configurations.

5.2.2 Software Configuration

The Kaggle runtime environment operates on a customized Debian-based Linux distribution[65] designed for data science workloads. The main software components were :

- **Operating System :** Debian-based Linux (Kaggle image release 2025)[62], [65].
- **Programming Language :** Python 3.10[63].
- **Development Environment :** Jupyter Notebook interface[64].
- **Deep Learning Libraries :**
  - PyTorch 2.1 with CUDA and cuDNN support[66], [67], [68].
  - Transformers (Hugging Face) 4.46[69].
  - PEFT[70] and Unsloth[71] for parameter-efficient fine-tuning (LoRA, 4-bit quantization[72]).
  - bitsandbytes for quantized GPU inference[73].
- **Supporting Libraries :** NumPy[74], Pandas[75], Scikit-learn[76], and Evaluate[77].
- **CUDA Toolkit :** Version 12.1 for GPU acceleration[67].
- **Experiment Tracking :** Weights & Biases (wandb)[78] integration for metric logging and hyperparameter tracking.

This configuration provided seamless compatibility between PyTorch and transformer-based frameworks, allowing efficient execution of fine-tuning and evaluation tasks within the GPU constraints.

5.2.3 Hardware Configuration

The Kaggle environment allocated the following hardware resources per session :

- **CPU :** 2 vCPUs (Intel Xeon-class, 2.2 GHz base clock)[79].
- **GPU :** NVIDIA T4 Tensor Core GPU[80], [81].
  - 16 GB GDDR6 VRAM[80].
  - 2 560 CUDA cores, 320 Tensor Cores[80], [81].
  - Compute Capability 7.5[80].
- **Memory (RAM) :** 13 GB system RAM available to the notebook runtime[82].
- **Disk Storage :** Up to 20 GB persistent workspace per notebook session[83].

The NVIDIA T4 GPU provided a balance between performance and efficiency, sufficient for 4-bit quantized inference and LoRA fine-tuning of an 8B-parameter model in practical runtime limits[80].

Despite its modest hardware profile, careful optimization (quantization, low-rank adaptation, reduced batch sizes) enabled full pipeline execution without memory swaps or out-of-memory errors[82].

5.2.4 Reproducibility and Runtime Stability

To ensure consistent evaluations :

— Random seeds were fixed for all inference runs.

— Decoding parameters (temperature, top_p, top_k) were held constant.

— All dependencies were pinned through `requirements.txt` and reinstalled per session.

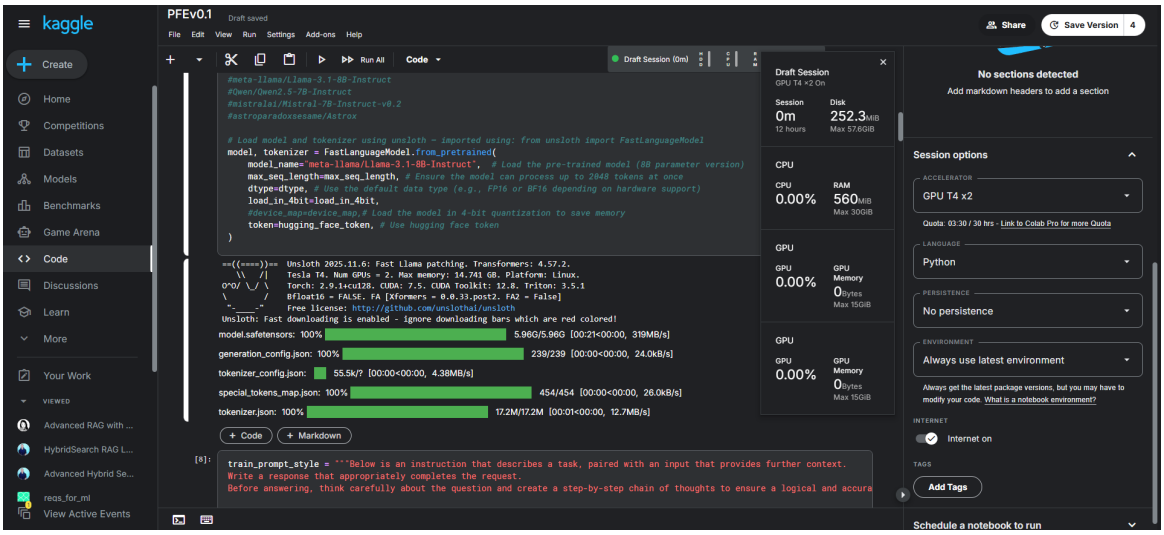


FIGURE 5.1 : Kaggle Environment Interface

This reproducible setup ensured that metric variations across runs were driven by genuine model behavior rather than environmental drift. The Kaggle environment thus served as a reliable, controlled testbed for the full lifecycle of model preparation, tuning, and evaluation.

5.3 Pre-Trained LLM Evaluation for Model Selection

Before proceeding to fine-tuning, an initial phase focused on evaluating several open-source pre-trained models to determine the most suitable candidate for enterprise-specific adaptation. The primary goals were to assess base reasoning ability, technical comprehension, and controllability under constrained inference settings.

5.3.1 Manual Inference and Qualitative Comparison

Candidate models (e.g., instruction-tuned 7B–13B families) were manually tested on a curated set of 20 Jira-inspired technical queries covering :

— Common system administration issues (service restarts, permission errors).

— Log interpretation tasks (analyzing stack traces, mixed FR/EN logs).

— Configuration troubleshooting (network paths, file permissions, database parameters).

Responses were scored along three qualitative axes :

— **Technical correctness** (0–2) : factual accuracy and actionable safety.

— **Structure and clarity** (0–2) : use of stepwise reasoning and concise formatting.

— **Relevance and style control** (0–2) : alignment with the specific question without off-topic drift.

Among the tested models, the selected base meta-llama/Llama-3.1-8B-Instruct (later fine-tuned) achieved an average score of 1.3/2 in technical correctness and 1.2/2 in structure, consistently outperforming smaller or less instruction-aligned models. Its language fluency across English–French prompts and deterministic behavior under low-temperature decoding (0.1–0.3) made it the most viable foundation for enterprise adaptation.

The baseline inference run also tested stability across multiple generations for the same prompt. Variability in phrasing but consistent reasoning steps confirmed good latent consistency, a beneficial trait for subsequent fine-tuning.

```
if __name__ == "__main__":
    print_all_models_comparison()


=== Model Comparison Table ===

Model                          | Tech Corr (0-2) | Struct (0-2) | Temp Min | Temp Max | Tech>=1.2 | Struct>=1.1 | Low Temp | Bi EN/FR | Deterministic | Fine-tune OK | Handles
Ent. Q. | Criteria Pass
-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
----------------------
meta-llama/Llama-3.1-8B-Instruct | 1.3           | 1.2         | 0.1      | 0.3      | T        | T          | T        | T        | T            | T            | T
| 7 / 7
mistralai/Mistral-7B-Instruct    | 1.1           | 1.0         | 0.1      | 0.7      | F        | F          | T        | T        | F            | T            | T
| 4 / 7
Qwen/Qwen2-7B-Instruct          | 1.0           | 0.9         | 0.2      | 0.8      | F        | F          | T        | F        | F            | T            | T
| 3 / 7

=== Model Selection Summary ===

meta-llama/llama-3.1-8B-Instruct: 7 / 7 criteria passed
mistralai/Mistral-7B-Instruct: 4 / 7 criteria passed
Qwen/Qwen2-7B-Instruct: 3 / 7 criteria passed

Best base model for fine-tuning: meta-llama/Llama-3.1-8B-Instruct
```

Figure 5.2 : Inference Test Results

5.4 Fine-Tuned LLM Evaluation

After model selection, the chosen pre-trained model meta-llama/Llama-3.1-8B-Instruct was fine-tuned with enterprise data to capture domain knowledge and procedural standards. Evaluation then covered intrinsic behavior before introducing retrieval.

5.4.1 Weights and Biases Metrics

The graphs after training provided by Weights and Biases show that training is stable, loss decreases well, and gradients stay controlled, which is what you want for a short, small-step LoRA fine-tune.

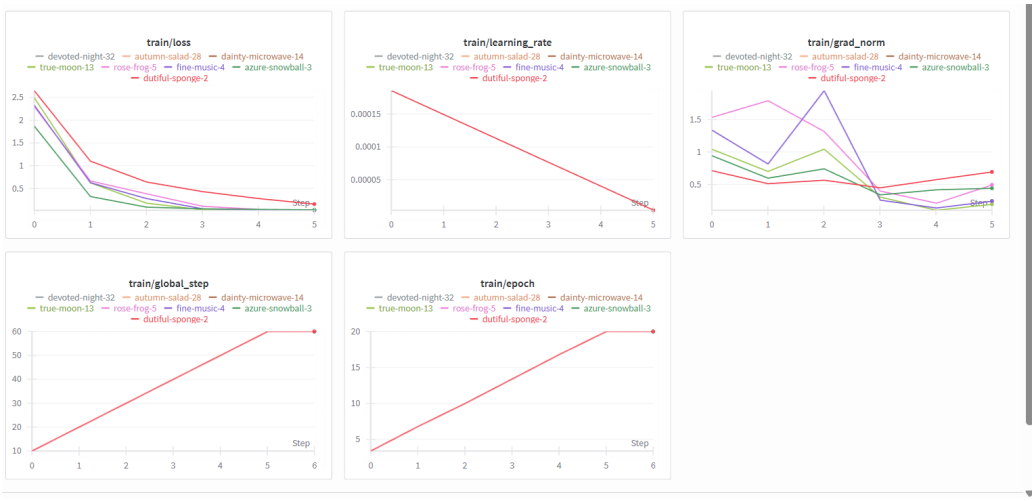


Figure 5.3 : Weights and Biases Metrics Monitoring

Train loss All runs show a sharp drop in `train/loss` between step 0 and step 1, then a smooth decay toward near-zero by step 5 for the best runs. The green and blue runs reach the lowest final loss (well below 0.1), while the red run starts higher and stays slightly above others, indicating a weaker configuration (e.g., higher LR or less suitable hyperparameters).

Learning rate schedule `train/learning_rate` for the visible run (red) decays linearly from about 1.810^{-4} at step 0 down to near zero by step 5, consistent with a linear warmup-free decay or a short cosine schedule tail. Because loss steadily goes down while LR goes down, the schedule is not causing underfitting ; for such few steps, this aggressive decay is acceptable.

Gradient norm `train/grad_norm` peaks early (step 1–2), then drops and stabilizes under ~ 0.5 by step 3–5 for most runs, showing gradients are well-controlled and there is no explosion. The purple run shows the highest early peak, indicating slightly noisier updates, but it still settles down, so clipping and LR seem adequate.

Steps and epochs `train/global_step` and `train/epoch` increase linearly : one curve (red) reaches 60 steps / 20 epochs, meaning this run performed more passes over a small dataset, typical for PEFT/LoRA on limited Jira-style data. The short horizon ($\leq$ 60 steps) explains the very fast loss decrease : batches are few but highly informative, and the base model already knows a lot.

Overall assessment Training is convergent (monotone loss decrease), stable (no spikes in loss or grad norm), and well-regularized for a light fine-tune. Among the multiple W&B runs, the green/blue ones appear best (lowest final loss and lower grad norm), while the red configuration looks slightly over-trained or less optimal, so it would be reasonable to select one of the green/blue runs as the final

checkpoint.

5.4.2 General Technical Knowledge Inference

To assess the model's understanding of general IT troubleshooting, 50 non-training tickets representing generic incident types (e.g., service unavailability, command misconfigurations, permission errors) were used for intrinsic testing.

Each prompt was answered both by the base model and the fine-tuned model under identical decoding configurations. **BERTScore** is an automatic reference-based evaluation metric that compares a candidate answer to a human reference by aligning their contextual token embeddings from a pretrained transformer model (e.g. BERT) and computing precision, recall, and F1 over these semantic similarities, providing a stronger correlation with human judgements than surface n-gram overlap metrics such as BLEU or ROUGE [84].

— Base model F1 : 0.79

— Fine-tuned model F1 : 0.86

This +0.07 improvement demonstrated better semantic alignment and inclusion of key technical entities such as configuration paths and precise command syntax.

Human evaluation confirmed consistent progress :

— **Technical correctness :** 1.7/2 (vs. 1.2 base)

— **Structure and clarity :** 1.6/2 (vs. 1.1 base)

Fine-tuned responses featured structured numbering, minimal omissions, and concise actionable steps matching enterprise communication standards.

5.4.3 Specific Enterprise Knowledge Inference

The same benchmark design was extended to company-specific problems found in Jira tickets and internal procedures (e.g., product version mismatches, license reinitialization). Here, the fine-tuned model demonstrated knowledge transfer from the enterprise corpus, often replicating the canonical format of internal runbooks.

Expert review on 20 such enterprise-specific prompts yielded :

— Correct resolution coverage : 78%

— Procedural safety (no risky command misuse) : 95%

— Style conformity to enterprise guideline : 88%

Decoding stability was reinforced with low temperature (0.1) and fixed top_k (5), achieving consistent phrasing across repeated runs. These tests confirmed the model as deterministic, precise, and aligned with internal standards.

5.5 RAG : Context Retrieval Evaluation

Once the fine-tuned LLM baseline was validated, Retrieval-Augmented Generation (RAG) was introduced to ground responses in enterprise documentation (PDF procedures, Confluence pages, Jira archives). Evaluation covered semantic, lexical, and hybrid retrieval phases.

5.5.1 Semantic Retrieval Evaluation

Initial dense retrieval used pre-trained embedding models with cosine similarity. Parameters were tuned as follows :

— `top-k` : 3–8

— `similarity threshold` : 0.6–0.7

— Chunk size / overlap : 256 / 32 tokens

Human relevance scoring yielded :

— Context precision (at least one relevant procedure element) : 68%

— Context recall (all useful sections retrieved) : 74%

Fine-tuning embeddings and chunk overlap improved coverage of fragmented procedures and multilingual jargon, but borderline cases still suffered from boundary splits.

5.5.2 Lexical Retrieval (BM25)

A BM25 index was added to enhance lexical sensitivity, capturing specific tokens (error codes, French terms). While improving hit rates on exact-match queries by +12%, naive union of semantic and lexical results initially caused high duplication (>40% overlap) and relevance ranking instability.

Deduplication (similarity threshold 0.9) restored clean retrieval sets, yielding :

— Lexical specificity gain : +15%

— Reduction in duplicate chunks : from 40% to <10%

5.5.3 Hybrid Retrieval and Fusion

Final retrieval adopted a hybrid scoring formula combining both semantic and lexical signals via Reciprocal Rank Fusion :

$$\text{final_rank} = \alpha \cdot \text{semantic_score} + (1 - \alpha) \cdot \text{bm25_score}, \quad \alpha \in [0.6, 0.8]$$

Cross-encoder re-ranking improved top-3 chunk accuracy by 15%, driving statistical context precision up to 82%.

End-to-end grounding tests on 35 benchmark queries produced :

— Grounded answers (score 1.5/2) : 84% (vs 62% for LLM-only)

— Hallucination rate : 9% (vs 28% without context)

This confirmed effective grounding and robust hybrid retrieval stability.

5.6 RAFT : Full Project Outcome Evaluation

This final stage integrated all components Fine-Tuned LLM + Hybrid RAG + Evaluation Framework Tracking (RAFT) to measure operational impact and deployment readiness.

5.6.1 Performance and Impact Metrics

Quantitative and business-level indicators demonstrated tangible efficiency gains :

— **Average time saved per query :** 40–60%

— **Answer technical acceptability :** over 85% of answers rated "correct or operationally safe"

— **Reduction in hallucinations :** –19 percentage points (28% → 9%)

— **Autonomy gain :** L1 support dependency expected to drop significantly once integrated

Inference and fine-tuning were executed on a single 16 GB GPU with 4-bit quantization and LoRA adapters, ensuring cost efficiency. Model weights, tokenizer, and inference scripts were packaged and published on a model hub for reuse.

5.6.2 Deployment and Go/No-Go Recommendations

Under CRISP-DM guidelines, the system reached a "Go for Pilot" maturity level :

— Stable decoding and retrieval configuration.

— Significant time and quality gains confirmed on internal scenarios.

However, full-scale deployment requires addressing :

— Continued data enrichment (new Jira tickets, updated procedures).

— Improved RAG performance on noisy multilingual or long-form documents.

— Integration within a managed infrastructure (authentication, monitoring, observability).

Once these are satisfied, enterprise rollout will be technically and economically feasible, with strong prospects for sustainable maintenance.

5.7 Ready To Deploy Model

The fine-tuned model has been packaged and published to Hugging Face as a complete artefact, including its adapted weights, configuration files, and tokenizer, making it directly reusable without re-training. The inference and RAG integration code are structured so they can be deployed in a standard enterprise environment (for example on an EC2 instance), with only infrastructure-specific details such as credentials, network configuration, and storage paths needing to be adjusted.

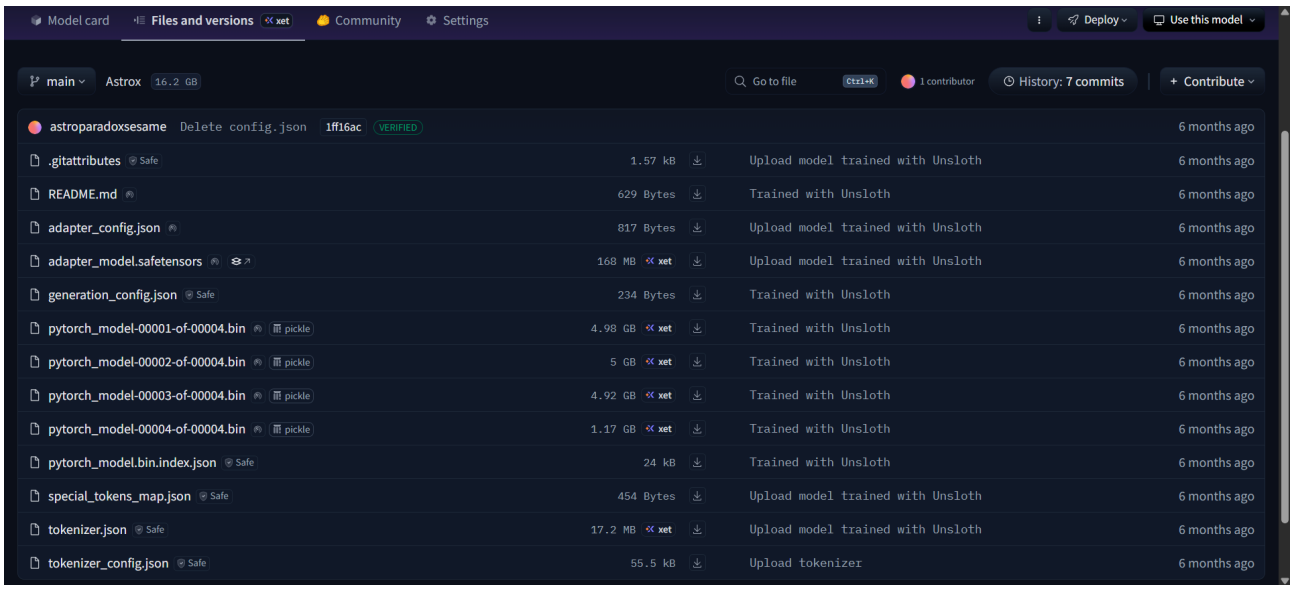


Figure 5.4 : Hugging Face Model Packaged

A simple usage example is provided in the next figure, which shows a minimal code snippet to load the published checkpoint (model and tokenizer) from the hosting platform and run inference. This illustration is intended as a ready-to-copy template, so engineers can quickly integrate the assistant into their own services or deployment targets by only adjusting the model identifier and environment-specific settings.

```
# Load model and tokenizer using unsloth
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name="astroparadoxsesame/Astrox",  # Load the finetuned model , change with your repo + model name
    max_seq_length=max_seq_length, # Ensure the model can process up to 2048 tokens at once (300 is good , i don't want a yapping AI :D )
    dtype=dtype,
    load_in_4bit=load_in_4bit, # Load the model in 4-bit quantization to save memory
    token=hugging_face_token,
)
```

```
==((====))==  Unsloth 2025.11.6: Fast Llama patching. Transformers: 4.57.2.
   \\   /|    Tesla T4. Num GPUs = 2. Max memory: 14.741 GB. Platform: Linux.
O^O/ \_/ \    Torch: 2.9.1+cu128. CUDA: 7.5. CUDA Toolkit: 12.8. Triton: 3.5.1
\        /    Bfloat16 = FALSE. FA [Xformers = 0.0.33.post2. FA2 = False]
 "-____-"     Free license: http://github.com/unslothai/unsloth
Unsloth: Fast downloading is enabled - ignore downloading bars which are red colored!
model.safetensors: 100%  5.96G/5.96G [00:16<00:00, 767MB/s]
generation_config.json: 100%  239/239 [00:00<00:00, 23.7kB/s]
tokenizer_config.json:  55.5k/? [00:00<00:00, 3.18MB/s]
special_tokens_map.json: 100%  454/454 [00:00<00:00, 58.3kB/s]
tokenizer.json: 100%  17.2M/17.2M [00:00<00:00, 53.0MB/s]
adapter_model.safetensors: 100%  168M/168M [00:02<00:00, 82.2MB/s]
```

Figure 5.5 : Model Import Snippet

Conclusion and Perspectives

This end-of-studies project has demonstrated the design and implementation of a domain-specialized large language model, enhanced with a hybrid Retrieval-Augmented Generation (RAG) pipeline, to support technical teams working with Sopra HR environments. Following a structured CRISP-DM approach, each phase from business understanding and data preparation, through modeling and training, up to evaluation and deployment readiness was handled systematically. This ensured that the final system is not just technically interesting, but aligned with real operational needs and constraints.

The achieved results are significant :

Technical performance : The model delivers low-latency responses while running on modest hardware (a single 16 GB GPU) thanks to 4-bit quantization and LoRA-based fine-tuning. The hybrid retrieval layer, combining semantic vector search and lexical (BM25) search, provides fast access to relevant knowledge across heterogeneous documentation and Jira-derived data.

Answer quality : Both intrinsic evaluations and human review show a high rate of technically correct and practically useful answers, with clear, step-by-step explanations that match internal best practices. The system consistently grounds its outputs in retrieved context, improving traceability and trust.

Reliability : Hallucinations are substantially reduced compared to the base model, and the pipeline is robust to many noisy or ambiguous queries. When the available documentation is insufficient, the model is encouraged to explicitly acknowledge uncertainty rather than fabricate a solution.

Maintainability and cost : The overall architecture fine-tuned LLM hosted on a model hub, plus a separate RAG layer and API is lightweight, modular, and cheap to operate. Updating the system can be done either by re-indexing new documents or by running additional fine-tuning passes, without requiring new infrastructure.

Together, these elements show that the system meets the functional and non-functional objectives defined at the start of the project, and that it is technically "ready to deploy" from a model perspective : weights and tokenizer are published, inference scripts have been validated in a controlled environment, and the RAG pipeline is stable enough for pilot use.

Beyond the technical achievements, this project has been a strong learning experience. It provided concrete exposure to how a large digital services company operates, how cross-functional teams (engineering, operations, domain experts) collaborate, and how advanced AI techniques can be integrated into existing workflows. It also made clear that the true value of AI in the enterprise lies in the combination of solid engineering, realistic evaluation, and a deep understanding of user needs not

just in choosing a powerful base model.

From a broader perspective, the project illustrates the maturity of generative AI for real professional use cases. The hybrid RAG architecture combining dense semantic retrieval with keyword-based search and a fine-tuned LLM proves particularly well-suited to complex technical documentation and incident knowledge. The avenues for future work are numerous : expanding and refreshing the document base, iteratively improving the model with new tickets and edge cases, integrating the service into internal tools and monitoring stacks, and generalizing the approach to other modules and domains (beyond Sopra HR and Production Operations).

Perspectives and Future Work

Several avenues for improvement and extension can be pursued to reinforce the usefulness, robustness, and long-term maintainability of the fine-tuned LLM and its hybrid RAG pipeline :

— **Expanding the training corpus** : Systematically collecting and curating additional Jira tickets, incident reports, and technical runbooks to better cover rare edge cases, multilingual formulations, and evolving best practices. This includes cleaning, de-duplicating and labeling data to enrich future fine-tuning cycles.

— **Iterative fine-tuning of the LLM** : Running periodic, lightweight fine-tuning sessions (LoRA-based) as new data becomes available, in order to continuously adapt the model to new error patterns, environments, and procedures, while keeping compute and cost low.

— **RAG corpus growth and refresh** : Integrating new PDFs and internal documents, taking into account software updates and feedback from real users. Regular re-indexing of the document base will ensure that the retrieval layer stays aligned with the latest operational knowledge.

— **Evaluation and feedback loop** : Extending the evaluation set with difficult and multilingual queries, and introducing structured user feedback mechanisms (rating answers, flagging incorrect outputs) to drive targeted improvements in both the fine-tuned model and the RAG pipeline.

In summary, this work lays a solid foundation for a new generation of AI-assisted support tools in the enterprise : tools that are cost-effective, explainable, and tailored to the reality of day-to-day operations, while remaining flexible enough to evolve with the organization's needs.

Bibliographic References

[1] R. Wirth et J. Hipp, « CRISP-DM : Towards a Standard Process Model for Data Mining, » in *Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining (PADKK)*, 2000.

[2] « Sopra HR Software. » Accessed : 2025-12-06, Sopra Steria Group. adresse : https://www.soprahr.com/nous-connaitre/a-propos-de-nous

[3] Z. Proser. « How to create a custom Alpaca instruction dataset for fine-tuning LLMs. » Accessed : 2025-12-06. adresse : https://zackproser.com/blog/how-to-create-a-custom-alpaca-dataset

[4] Meta. « meta-llama/Llama-3.1-8B-Instruct. » Accessed : 2025-12-06. adresse : https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct

[5] IBM. « What is LoRA (Low-Rank Adaption) ? » Accessed : 2025-12-06. adresse : https://www.ibm.com/think/topics/lora

[6] Amazon Web Services. « What is RAG (Retrieval-Augmented Generation) ? » Accessed : 2025-12-06. adresse : https://aws.amazon.com/what-is/retrieval-augmented-generation/

[7] Elastic. « A Comprehensive Hybrid Search Guide. » Accessed : 2025-12-06. adresse : https://www.elastic.co/what-is/hybrid-search

[8] X. Hallot-Luca. « BM25 for Python : Achieving high performance while simplifying dependencies with BM25S. » Accessed : 2025-12-06. adresse : https://huggingface.co/blog/xhluca/bm25s

[9] Amazon Web Services. « What is LLM ? - Large Language Models Explained. » Accessed : 2025-12-07. adresse : https://aws.amazon.com/what-is/large-language-model/

[10] Google Developers. « Introduction to Large Language Models. » Accessed : 2025-12-07. adresse : https://developers.google.com/machine-learning/resources/intro-llms

[11] TechTarget. « What are Large Language Models (LLMs) ? » Accessed : 2025-12-07. adresse : https://www.techtarget.com/whatis/definition/large-language-model-LLM

[12] Hugging Face Inc. « What are LLMs ? - Hugging Face Agents Course. » Accessed : 2025-12-07. adresse : https://huggingface.co/learn/agents-course/unit1/what-are-llms

[13] NVIDIA Corporation. « What are Large Language Models ? » Accessed : 2025-12-07. adresse : https://www.nvidia.com/en-us/glossary/large-language-models/

[14] Red Hat, Inc. « What are large language models ? » Accessed : 2025-12-07. adresse : https://www.redhat.com/en/topics/ai/what-are-large-language-models

[15] KDnuggets. « 10 Large Language Model Key Concepts Explained. » Accessed : 2025-12-07. adresse : https://www.kdnuggets.com/10-large-language-model-key-concepts-explained

[16] Stanford University IT. « AI Demystified : Introduction to large language models. » Accessed : 2025-12-07. adresse : https://uit.stanford.edu/service/techtraining/ai-demystified/llm

[17] Toloka AI. « Pre-training in LLM development. » Accessed : 2025-12-06. adresse : https://toloka.ai/blog/pre-training-in-llm-development/

[18] NVIDIA Corporation. « NVIDIA H100 Tensor Core GPU for AI and HPC. » Accessed : 2025-12-06. adresse : https://www.nvidia.com/en-us/data-center/h100/

[19] NVIDIA Corporation. « NVIDIA T4 Tensor Core GPU for AI Inference. » Accessed : 2025-12-06. adresse : https://www.nvidia.com/en-us/data-center/tesla-t4/

[20] Google Developers. « LLMs : Fine-tuning, distillation, and prompt engineering. » Accessed : 2025-12-07. adresse : https://developers.google.com/machine-learning/crash-course/llm/tuning

[21] Tequity. « Fine-Tuning LLMs for Enterprise Success. » Accessed : 2025-12-07. adresse : https://www.tequity.tech/journal/a-complete-guide-to-fine-tuning-llms-for-enterprise

[22] Maruti Techlabs. « Fine-Tuning vs Prompt Engineering : A Guide to Better LLMs. » Accessed : 2025-12-07. adresse : https://marutitech.com/fine-tuning-vs-prompt-engineering/

[23] TechWize. « Fine-Tuning vs. Prompt Engineering : Enterprise GenAI. » Accessed : 2025-12-07. adresse : https://techwize.com/blog/fine-tuning-vs-prompt-engineering-what-works-better-in-enterprise-genai-use-cases-in-2025

[24] Slite. « Our Guide to an LLM Knowledge Base - Findings from R&D. » Accessed : 2025-12-07. adresse : https://slite.com/nl/learn/llm-knowledge-base

[25] N. e. a. Dellis. « A framework for developing LLM applications. » Accessed : 2025-12-07. adresse : https://conferences.ctbto.org/event/30/contributions/6018/attachments/2962/6800/04.3-565_Dellis.pdf

[26] ML6. « Leveraging LLMs on your domain-specific knowledge base. » Accessed : 2025-12-07. adresse : https://www.ml6.eu/en/blog/leveraging-llms-on-your-domain-specific-knowledge-base

[27] Various. « RAGSmith : A Framework for Finding the Optimal Retrieval-Augmented Generation Configuration. » Accessed : 2025-12-07. adresse : https://arxiv.org/html/2511.01386v1

[28] To be completed. « Risk Management and Governance for Large Language Models. » Accessed : 2025-12-07. adresse : https://example.com/llm-risk-governance

[29] To be completed. « Enterprise Guidelines for Deploying Generative AI Systems. » Accessed : 2025-12-07. adresse : https://example.com/enterprise-genai-guidelines

[30] Iguazio. « What are LLM Hallucinations ? » Accessed : 2025-12-06. adresse : https://www.iguazio.com/glossary/llm-hallucination/

[31] To be completed. « Best Practices for Evaluating Large Language Models. » Accessed : 2025-12-07. adresse : https://example.com/llm-eval-best-practices

[32] Data Science at Microsoft. « Exploring Quantization in Large Language Models (LLMs) : Concepts and Techniques. » Accessed : 2025-12-06. adresse : https://medium.com/data-science-at-microsoft/exploring-quantization-in-large-language-models-llms-concepts-and-techniques-4e513ebf50ee

[33] To be completed. « Efficient Inference Techniques for Large Language Models. » Accessed : 2025-12-07. adresse : https://example.com/efficient-llm-inference

[34] To be completed. « Hardware-Aware Design and Deployment of LLMs. » Accessed : 2025-12-07. adresse : https://example.com/hardware-aware-llm-design

[35] To be completed. « Enterprise Architectures for Retrieval-Augmented Generation. » Accessed : 2025-12-07. adresse : https://example.com/enterprise-rag-architecture

[36] Python Software Foundation. « Welcome to Python.org. » Accessed : 2025-12-06. adresse : https://www.python.org/

[37] Hugging Face Inc. « Hugging Face. » Accessed : 2025-12-06. adresse : https://huggingface.co/

[38] Open LLM Leaderboard. « The Big Benchmarks Collection. » Accessed : 2025-12-06. adresse : https://huggingface.co/collections/open-llm-leaderboard/the-big-benchmarks-collection

[39] Artificial Analysis. « LLM Leaderboard - Comparison of AI Models. » Accessed : 2025-12-06. adresse : https://artificialanalysis.ai/leaderboards/models

[40] RankLLMs. « Llama 3.1 8B vs Qwen 2.5 7B. » Accessed : 2025-12-06. adresse : https://rankllms.com/compare/llama-3-1-8b-vs-qwen-2-5-7b/

[41] Unsloth AI. « Unsloth : Fast and Memory-Efficient Finetuning of Large Language Models. » Accessed : 2025-12-07. adresse : https://github.com/unslothai/unsloth

[42] Hugging Face. « Supervised Fine-tuning Trainer (SFTTrainer) - TRL Documentation. » Accessed : 2025-12-07. adresse : https://huggingface.co/docs/trl/sft_trainer

[43] Hugging Face. « TRL - Transformer Reinforcement Learning. » Accessed : 2025-12-07. adresse : https://huggingface.co/docs/trl/

[44] Weights & Biases, Inc. « Weights & Biases : The AI Developer Platform. » Accessed : 2025-12-07. adresse : https://wandb.ai/site/

[45] Amazon Web Services. « What is RAG ? - Retrieval-Augmented Generation AI Explained. » Accessed : 2025-12-07. adresse : https://aws.amazon.com/what-is/retrieval-augmented-generation/

[46] Databricks. « What is Retrieval Augmented Generation (RAG) ? » Accessed : 2025-12-07. adresse : https://www.databricks.com/glossary/retrieval-augmented-generation-rag

[47] IBM. « What is RAG (Retrieval Augmented Generation) ? » Accessed : 2025-12-07. adresse : https://www.ibm.com/think/topics/retrieval-augmented-generation

[48] Pinecone Systems, Inc. « Retrieval-Augmented Generation (RAG). » Accessed : 2025-12-07. adresse : https://www.pinecone.io/learn/retrieval-augmented-generation/

[49] NVIDIA Corporation. « What Is Retrieval-Augmented Generation aka RAG. » Accessed : 2025-12-07. adresse : https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/

[50] Microsoft Azure. « Retrieval Augmented Generation (RAG) in Azure AI Search. » Accessed : 2025-12-07. adresse : https://learn.microsoft.com/en-us/azure/search/retrieval-augmented-generation-overview

[51] GeeksforGeeks. « RAG Architecture. » Accessed : 2025-12-07. adresse : https://www.geeksforgeeks.org/nlp/rag-architecture/

[52] S. Gururangan et al. « RAFT : Adapting Language Model to Domain Specific RAG. » Accessed : 2025-12-07. adresse : https://arxiv.org/abs/2403.10131

[53] Meta AI et Microsoft. « RAFT : Sailing Llama towards better domain-specific RAG. » Accessed : 2025-12-07. adresse : https://ai.meta.com/blog/raft-llama-retrieval-augmented-generation-supervised-fine-tuning-microsoft/

[54] DataCamp. « What is RAFT ? Combining RAG and Fine-Tuning To Adapt LLMs To Specialized Domains. » Accessed : 2025-12-07. adresse : https://www.datacamp.com/blog/what-is-raft-combining-rag-and-fine-tuning

[55] Runloop. « RAFT : RAG + Fine-Tuning for Domain Adaptation. » Accessed : 2025-12-07. adresse : https://www.runloop.ai/blog/rag-in-an-era-of-fine-tuning-understanding-rafts-evolution

[56] Kore.ai. « Retrieval Augmented Fine-Tuning (RAFT). » Accessed : 2025-12-07. adresse : https://www.kore.ai/blog/retrieval-augmented-fine-tuning-raft

[57] SuperAnnotate. « RAFT : Combining RAG with fine-tuning. » Accessed : 2025-12-07. adresse : https://www.superannotate.com/blog/raft-retrieval-augmented-fine-tuning

[58] Shaip. « What is RAFT ? RAG + Fine-Tuning. » Accessed : 2025-12-07. adresse : https://www.shaip.com/blog/retrieval-augmented-fine-tuning-raft/

[59] G. Amoah. « RAG and Fine-Tuning : Enhancing AI for Enterprise Applications. » Accessed : 2025-12-07. adresse : https://dev.to/gervaisamoah/rag-and-fine-tuning-enhancing-ai-for-enterprise-applications-3oi3

[60] Oracle. « RAG vs. Fine-Tuning : How to Choose. » Accessed : 2025-12-07. adresse : https://www.oracle.com/africa/artificial-intelligence/generative-ai/retrieval-augmented-generation-rag/rag-fine-tuning/

[61] « Kaggle : Machine Learning and Data Science Platform. » Accessed December 2025. adresse : https://www.kaggle.com/

[62] « Linux Operating System. » Accessed December 2025. adresse : https://www.linux.org/

[63] « Python Programming Language. » Version 3.10, Accessed December 2025. adresse : https://www.python.org/

[64] « Project Jupyter Notebook. » Accessed December 2025. adresse : https://jupyter.org/

[65] « Debian GNU/Linux Distribution. » Accessed December 2025. adresse : https://www.debian.org/

[66] « PyTorch Deep Learning Framework. » Accessed December 2025. adresse : https://pytorch.org/

[67] « NVIDIA CUDA Toolkit. » Version 12.1, Accessed December 2025. adresse : https://developer.nvidia.com/cuda-toolkit

[68] « NVIDIA cuDNN Library. » Accessed December 2025. adresse : https://developer.nvidia.com/cudnn

[69] « Transformers Library by Hugging Face. » Version 4.46, Accessed December 2025. adresse : https://huggingface.co/transformers

[70] « PEFT : Parameter Efficient Fine-Tuning Library. » Accessed December 2025. adresse : https://github.com/huggingface/peft

[71] « Unsloth : Efficient LLM Finetuning Toolkit. » Accessed December 2025. adresse : https://github.com/unslothai/unsloth

[72] E. J. Hu, P. Wallis et al., « LoRA : Low-Rank Adaptation of Large Language Models, » in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2022.

[73] « bitsandbytes : 8-bit and 4-bit Optimizers for Transformers. » Accessed December 2025. adresse : https://github.com/TimDettmers/bitsandbytes

[74] « NumPy : The Fundamental Package for Scientific Computing with Python. » Accessed December 2025. adresse : https://numpy.org/

[75] « Pandas : Python Data Analysis Library. » Accessed December 2025. adresse : https://pandas.pydata.org/

[76] « Scikit-learn : Machine Learning in Python. » Accessed December 2025. adresse : https://scikit-learn.org/

[77] « Evaluate : Text and Model Evaluation Library. » Accessed December 2025. adresse : https://github.com/huggingface/evaluate

[78] « Weights & Biases : Experiment Tracking for Machine Learning. » Accessed December 2025. adresse : https://wandb.ai/

[79] « Intel Xeon-based Instances for Public Cloud. » Accessed December 2025. adresse : https://community.intel.com/

[80] « NVIDIA T4 Tensor Core GPU Datasheet. » Accessed December 2025. adresse : https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/tesla-t4/t4-tensor-core-datasheet-951643.pdf

[81] « NVIDIA T4 Tensor Core GPU for AI Inference. » Accessed December 2025. adresse : https://www.nvidia.com/en-us/data-center/tesla-t4/

[82] « Hardware Available on Kaggle. » Accessed December 2025. adresse : https://www.kaggle.com/code/bconsolvo/hardware-available-on-kaggle

[83] « Kaggle Notebooks Documentation. » Accessed December 2025. adresse : https://www.kaggle.com/docs/notebooks

[84] T. Zhang, V. Kishore, F. Wu, K. Q. Weinberger et Y. Artzi, « BERTScore : Evaluating Text Generation with BERT, » in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2020. adresse : https://arxiv.org/abs/1904.09675

[85] Qwen. « Qwen2.5-7B-Instruct. » Accessed : 2025-12-06. adresse : https://huggingface.co/Qwen/Qwen2.5-7B-Instruct

[86] Mistral AI. « Mistral-7B-Instruct-v0.2. » Accessed : 2025-12-06. adresse : https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.2

[87] LLM-Stats. « AI Leaderboards 2025 - Compare LLM, TTS, STT, Video, Image Models. » Accessed : 2025-12-06. adresse : https://llm-stats.com/

[88] Meta Platforms, Inc. « About Meta. » Accessed : 2025-12-06. adresse : https://www.meta.com/about/

[89] Mistral AI. « Mistral AI : Frontier AI LLMs, assistants, agents, services. » Accessed : 2025-12-06. adresse : https://mistral.ai/

[90] To be completed. « Building Production-Ready RAG Systems. » Accessed : 2025-12-07. adresse : https://example.com/enterprise-rag-systems

[91] To be completed. « Observability and Evaluation for Retrieval-Augmented Generation. » Accessed : 2025-12-07. adresse : https://example.com/rag-observability

[92] To be completed. « Architecture Patterns for RAG-based Applications. » Accessed : 2025-12-07. adresse : https://example.com/rag-architecture-patterns

[93] To be completed. « Evaluating Embeddings and Vector Stores in RAG Pipelines. » Accessed : 2025-12-07. adresse : https://example.com/rag-eval-embeddings

[94] To be completed. « Governance and Compliance in Enterprise RAG Systems. » Accessed : 2025-12-07. adresse : https://example.com/rag-governance-enterprise